

Determining an Appropriate Mixing Time for Hybrid Artificial Reverberation by Analysis of the Fractal Dimension Profile of Room Impulse Responses

Thomas J. Conaty

Music & Media Technologies
School of Engineering
&
School of Drama, Film and Music
Trinity College Dublin



Submitted as part fulfilment for the degree of M.Phil.

2011

DECLARATION

I hereby declare that this thesis has not been submitted as an exercise for a degree at this or any other University and that it is entirely my own work.

I agree that the Library may lend or copy this thesis upon request.

Signed: _____

Date: _____

ABSTRACT

Hybrid artificial reverberation is a form of artificial reverberation that aims to take advantage of the acoustic realism of reverberation created through convolution with room impulse responses and the efficiency and versatility of reverberation created using filterbanks. In order to make best use of each type of reverberation, the point at which a room impulse response transitions from early to late reflections (the mixing time) must be known.

Since the mid-20th century, fractal geometry has provided a new language by which to describe and explore the mechanism of many natural and physical processes. It has facilitated advances in fields ranging from telecommunications to human-integrated technologies.

In this project, the Higuchi algorithm for calculating fractal dimension was used to produce fractal dimension profiles for eight room impulse responses. These room impulse responses were produced using Allen and Berkley's image-source method and were modelled on shoebox shaped rooms from a study looking at the perceptual evaluation of mixing time. The resulting fractal dimension profiles for the eight room impulse responses were analysed in terms of the length of time taken for the profile to reach a threshold determined by one of four different criteria, and the resulting values compared to results from the perceptual evaluation of the mixing time study. Through this analysis an equation calculating the appropriate mixing time for use in hybrid artificial reverberation was established. This equation produced an explained variance of 93.49% with the results for the perceptual evaluation of mixing time.

The results from this study show that analysis of the fractal dimension profile of room impulse responses offers a method for calculating the appropriate mixing time for hybrid artificial reverberation that is both simple in its implementation and of greater accuracy to previous methods.

ACKNOWLEDGEMENTS

This thesis was only made possible by the support by a number of generous people.

I would first like to thank my girlfriend Sophie Murray for her enduring encouragement and patience throughout this process, whose companionship was both an inspiration and sanctuary. To my parents, Anne and Tom, I owe a huge debt of gratitude, first and foremost for nurturing in me a deep appreciation for art and science and the desire to explore those worlds, as well as for the financial and moral support they have provided throughout this thesis. I would also like to thank my sister Julie for always bringing a smile to my face when the going was tough and the interest she showed in my work.

Furthermore, I am extremely grateful for the patience, time and guidance provided to me by my supervisors Course Director, Prof. Dermot Furlong and Gavin Kearney. Their expertise and knowledge brought structure to my ambitions and guided me down the unlit paths of my journey.

I must also thank Dr. Fionnuala Conway along with all the wonderful teaching staff and fellow students for making the last two years so worthwhile and enjoyable.

Finally I would like to thank all my friends and loved ones whose company and kind words have been the perfect antidote to my stress.

CONTENTS

1	INTRODUCTION	1
2	OVERVIEW OF REVERBERATION	5
2.1	Sabine's Reverberation Equation	5
2.2	Room Impulse Response	6
2.2.1	Mixing Time	7
2.2.2	Diffusion of Reflections in a Room	9
2.3	Digital Reverberation	9
2.3.1	Convolution	9
2.4	Artificial Digital Reverberation.....	10
2.4.1	Image Source Method.....	11
2.4.2	Algorithmic Artificial Reverberation.....	11
2.4.3	Filter Implementation in Algorithmic Artificial Reverberation	14
2.4.4	Feedback Delay Networks	17
2.5	Hybrid Artificial Reverberation	18
3	FRACTAL GEOMETRY	20
3.1	Development of Fractal Geometry	20
3.1.1	Fractal Dimension.....	22
3.1.2	Calculating the Fractal Dimension of Real-World Fractal Systems	24
4	PREDICTING THE MIXING TIME.....	27
4.1	Model Predictors	27
4.2	Empirically Derived Predictors	28
4.3	Perceptual Evaluation of Mixing Time Predictors	30
4.3.1	Method.....	30
4.3.2	Results.....	32

5	METHOD.....	38
5.1	Image Source Method	39
5.2	Fractal Dimension Profile of Room Impulse Responses.....	39
5.3	Analysis	41
6	RESULTS.....	43
6.1	Criteria I – IV	43
6.2	Correlation with Mean Perceptual Mixing Time	45
7	DISCUSSION & CONCLUSIONS	48
7.1	Further Work	49
8	BIBLIOGRAPHY	50
9	APPENDIX	53
9.1	MATLAB Code.....	53
9.1.1	FDthresh.m	53
9.1.2	HMW.m	54
9.1.3	hfd.m	55
9.1.4	Image Source Method Matlab code	56
9.1.5	Image Source Method GUI.....	58
9.2	Convolution	74
9.2.1	Direct Time Domain Convolution	74
9.2.2	Direct Frequency Domain Convolution.....	74
9.2.3	Block Convolution.....	75
9.3	Simulation Techniques	79
9.3.1	Image-Source Model – Mathematics.....	79
9.4	Artificial Digital Reverberation – Mathematics	81
9.4.1	Analysis Techniques of Digital Filters	81
9.5	The Comb and All-Pass Filters in Detail.....	84
9.5.1	The Comb Filter.....	84
9.5.2	All-pass Filter	85
9.5.3	Feedback Delay Networks.....	87

LIST OF FIGURES

Figure 2.2a: <i>Theoretical RIR showing distinct sections of direct sound, early and late reflections</i>	7
Figure 2.2.b: <i>A typical, real RIR of a room showing SPL variations about zero on the y-axis over time</i>	8
Figure 2.2c: <i>RIR created using the ISM [Allen & Berkley, 1979] showing relative, positive SPL variation on the y-axis over time</i>	8
Figure 2.4.1: <i>Image sources in the x-y plane. R represents the receiver position, the circle marked S is the original source with the other circles, and I_i being image-sources in adjacent mirror rooms, where $i = 1, 2, 3, 4$</i>	12
Figure 2.4.2.1a: <i>Block diagram and Impulse response of a comb filter (See Appendix for details)</i>	13
Figure 2.4.2.1b: <i>Frequency response of a comb filter</i>	13
Figure 2.4.2.2a: <i>Block diagram and impulse response of an all-pass filter</i>	13
Figure 2.4.2.2b: <i>Frequency response of all-pass filter</i>	13
Figure 2.4.3.1: <i>Schroeder Reverberator 1</i>	15
Figure 2.4.3.2: <i>Schroeder Reverberator 2</i>	15
Figure 2.4.3.3: <i>Moorer reverberators</i>	15
Figure 2.4.3.4: <i>Graphic representation of Gardiner's late reflections artificial digital reverberations algorithms for small, medium and large rooms</i>	16
Figure 2.4.3.5: <i>Block Diagram of Gardiner's reverberators for a large room</i>	16
Figure 2.4.4.1: <i>FDN of order $N=3$</i>	18
Figure 2.4.4.2: <i>A block diagram of Jot's reverberator employing a FDN with a tonal correction filter</i>	18
Figure 2.5: <i>Structure used in HAR [Murphy & Stewart, 2007]</i>	19
Figure 3.1a: <i>Iterations of the Koch Curve</i>	22
Figure 3.1b: <i>Visualisation of the Mandelbrot Set</i>	23
Figure 3.1.1: <i>Box-Counting FD method, employed on the coast of the UK</i>	24
Figure 3.1.2a: <i>Estimated FD against Theoretical FD for a number of FD algorithms</i>	25

Figure 3.1.2b: Cascade showing effect on FD for various algorithms, going from a random to sinusoid waveform over a short sample length. Note the Higuchi algorithm swiftly moving to a FD of 1 with little noise upon introduction of sinusoidal waveform.....	25
Figure 4.3.1.1: Floor plans for nine rooms from Lindau et al study.....	31
Figure 4.3.2: Mean perceptual mixing time, $t_{mp50\%}$, for the nine rooms with 95% confidence intervals.....	33
Figure 4.3.2.1. Linear regression analysis of the various model predictors and individual variables for $t_{mp50\%}$ (left cascade) and $t_{mp95\%}$ (right cascade) with 95% CI.	35
Figure 4.3.2.2. Linear regression analysis of the various empirically derived predictors for $t_{mp50\%}$ (left cascade) and $t_{mp95\%}$ (right cascade), with 95% CI.....	36
Figure 5.1a: GUI for ISM in MATLAB, showing typical setup to produce an RIR.....	40
Figure 5.1b: ISM generated RIR for Room 1 from Lindau study.....	40
Figure 5.2: Fractal Dimension Profile from RIR of Room 1 created using HMW.m., with a window size of 50 samples.....	41
Figure 5.3: FD profile for Room 1 including $t_{mp50\%}$ and thresholds for Criteria I – IV.....	42
Figure 6.2: Plots of $t_{mp50\%}$ against sample values from Criteria I – IV with regression equation and R^2 values.....	46
Figure 9.2.1: Block diagram representation of FIR filter for time domain convolution.....	75
Figure 9.2.3.1a: Input signal and RIR for Overlap and Add example.....	77
Figure 9.2.3.1b: Three output blocks from Overlap and Add example.....	77
Figure 9.2.3.1c: Output from Overlap and Add example.....	77
Figure 9.2.3.2a: Overlap and Save example showing input signal (divided into blocks, denoted by horizontal brackets) and RIR.....	77
Figure 9.2.3.2b Output blocks from Overlap and Save example. Samples to be discarded before summation shown in gray.....	78
Figure 9.2.3.2c: Output from Overlap and Save example.....	78
Figure 9.2.3.3a: Gardiner's zero-delay convolution algorithm.....	79
Figure 9.2.3.3b: Gardner's second proposed algorithm with a more practical decomposition of the impulse response with the corresponding convolution schedule.	80

Figure 9.4.1.2: <i>Block diagram of a comb filter</i>	83
Figure 9.4.1.5: <i>Zero-pole plot of transfer function in z-plane</i>	84
Figure 9.5.1a: <i>Block diagram and Impulse response of a comb filter</i>	86
Figure 9.5.1b: <i>Frequency response of comb filter</i>	86
Figure 9.5.1c: <i>Frequency response in z-plane</i>	86
Figure 9.5.2a: <i>Block diagram and Impulse response of an all-pass filter</i>	88
Figure 9.5.2b: <i>Frequency response of an all-pass filter</i>	88
Figure 9.5.3c: <i>Frequency response in z-plane</i>	88

LIST OF TABLES

Table 4.3.1.1: <i>Volume, RT and α_{ave} for nine rooms in Lindau et al 2010 study</i>	31
Table 4.3.2.2: <i>Summary of correlation results for $t_{mp50\%}$ and $t_{mp95\%}$ against predictors. Red indicates an R^2 below 40%, orange indicates an R^2 between 40% and 60%, yellow indicates an R^2 between 60% and 80%, and an R^2 over 80% is shown in green</i>	37
Table 6.1a: <i>Values of mean and standard deviation for the last 10% of the FD profiles of Rooms 1 – 8</i>	44
Table 6.1b: <i>Sample number (44100 samples s^{-1}) from the eight FD profiles for the four threshold criteria</i>	44
Table 6.2: <i>Linear regression equations and explained variance for Criteria I – IV</i>	46
Table 6.3: <i>Summary of correlation results for $t_{mp50\%}$ against predictors. Red indicates an R^2 below 40%, orange indicates an R^2 between 40% and 60%, yellow indicates an R^2 between 60% and 80%, and an R^2 over 80% is shown in green</i>	47

LIST OF ACRONYMS

Virtual Acoustic Environments (VAE)

Hybrid Artificial Reverberation (HAR)

Room Impulse Response (RIR)

Fractal Dimension (FD)

Functional Magnetic Resonance Imaging (fMRI)

Image Source Model (ISM)

Sound Pressure Level (SPL)

Reverberation Time (RT)

Fast Fourier Transform (FFT)

Discrete Fourier Transform (DFT)

Artificial Digital Reverberation (ADR)

Digital Signal Processing (DSP)

Hybrid Artificial Reverberation (HAR)

Infinite Impulse Response (IIR)

Finite Impulse Response (FIR)

Feedback Delay Network (FDN)

Confidence Intervals (CI)

Binaural Room Impulse Response (BRIR)

1 INTRODUCTION

The design of Virtual Acoustic Environments (VAE) of increasing sophistication is continually being sought to be implemented in conjunction with 3D visual environments. This has become increasingly important in various fields of research, as well as for special effects in the film industry and in the ever more sophisticated video game industry. As these 3D environments increase in visual sophistication, there is an expectation to provide an equally realistic acoustic environment. One factor that must be considered when constructing a VAE is the minimisation of the processing power required to create it. One way this can be achieved is through the use of hybrid artificial reverberation (HAR) [Primavera et al, 2010]. HAR aims to exploit the knowledge of a room's reflection profile (which is known as its room impulse response, RIR), and the limit of human perception of distinct reflections within a particular acoustic environment.

An RIR comprises three parts: The direct sound (the initial acoustic input into the room), the early reflections (which provide the listener with information on the geometry of the physical space) and the late reflections (the point at which the listener can no longer differentiate between reflections). The point at which the early reflections transition into the later reflections can either be described in terms of the mathematical behaviour of the system, known as the mixing time (t_{mix}) or in terms of how the auditory system perceives the behaviour of the system, known as the perceptual mixing time (t_{mp}). HAR uses a convolution of a recorded or mathematically modelled RIR with an input sound to generate the early reflections, and then generates the late reflections algorithmically using filters [Primavera et al, 2010; Murphy & Stewart, 2007]. The convolution of an input sound with an RIR produces the best possible reproduction of a room's reflection profile but suffers from being computationally expensive. Algorithmically generated reverberation is computationally inexpensive but can produce unnatural audible artefacts. To make the most of HAR by maximising sound quality while minimising computational load, it is vital that a method exists so that the t_{mix} can be precisely calculated.

Fractals are a geometric phenomenon regularly observed in nature, the mathematics of which have been exploited since the latter half of the 20th century to explain and explore a number of different of natural systems. Fractals are characterised as rough or fragmented shapes that are self-similar at various levels of magnification, and can be described by a dimension that is a non-integer value greater than its topological dimension. The fractal dimension (FD) of a fractal is a statistical quantity used to describe how completely that fractal fills space as the level of magnification used to view it is varied, and is a measure of how complicated the shape is [Mandelbrot, 1982]. A mathematical algorithm can be used for estimating the FD of real-world fractal systems as, invariably, the fractal will not be self-similar at all levels of magnification. The estimated FD describes how complex the fractal is, and hence how much space it appears to fill within a specific magnification range [Higuchi, 1988; Raghavendra & Dutt, 2010].

Over the past decade or so, the mathematical understanding of fractal systems has been exploited on a number of fronts; it has been used to vastly improve telecommunication systems and minimise the size of antennae [Hohlfield, 1999], to create computer-based musical instrument recognition systems [Gunasekaran & Revathy 2008], and also to detect changes in neural activity in the brain [Raghavendra & Dutt 2009]. Recently it has also been observed that how the eye tracks images on a screen, the paths animals take when foraging for food, and the aesthetic appeal of objects have a strict fractal dependence. In addition to this, images of a FD of around 1.5 can reduce the physiological response to stress by up to 60%, with functional magnetic resonance imaging (fMRI) experiments showing that these mid-range FD images activate distinct regions of the brain associated with the regulation of emotions such as happiness [Taylor, 2011].

These examples highlight the pervasive and fundamental role fractal systems play in our natural and physical environment. More specifically it exposes the fractal dependence on a number of cognitive systems within the human brain and opens up the possibility that the auditory perception of our environment may share a similar dependence.

For this investigation, the results from a paper detailing a study that evaluated t_{mp} for a range of typical acoustic environments [Lindau, 2010], were compared with time-varying fractal dimension profiles of modelled RIRs for these environments, with the intention of developing a method for predicting t_{mp} by analysis of a time-varying FD profile of an RIR. The paper also details a comprehensive, comparative study of previous methods for

predicting t_{mp} , the results of which are presented in Section 6.3 to demonstrate the effectiveness of the method developed in this investigation.

The modelled RIRs used for this investigation were created using the image-source method (ISM) [Allen & Berkley, 1979], which was able to make use of values provided in the paper for the average absorption coefficients (α_{ave}) and geometric dimensions of each room. The study examined nine rooms ranging in volume from 216m³ to 8500m³, with values for α_{ave} ranging from 0.17 to 0.36 and reverberation times ranging from 0.39seconds to 2.08seconds. From a series of listening tests these rooms were found to have mean perceptual mixing times ($t_{mp50\%}$) ranging from 37.5ms to 97.8ms. Eight of the nine rooms from this study were modelled to produce RIRs (see Chapter 5 for disqualification criteria of the ninth room).

A number of different algorithms are available for calculating the FD of real world systems. In this investigation the Higuchi algorithm was used due to its demonstrated accuracy against fractals of known FD, and its ability to handle small data sets [Higuchi, 1988; Raghavendra & Dutt, 2010]. FD profiles for the modelled RIRs were obtained using a ‘moving-window’ function written in MATLAB (*HMW.m*, see Appendix 1.2 for MATLAB code). This function works by travelling along the RIR one sample at a time, applying the Higuchi Algorithm¹ (*hfd.m*, see Appendix 1.3) to calculate the FD over the length of the window. This function produces a profile of how the FD of each RIR changes over time.

After producing FD profiles for the eight rooms, they were then individually compared to their respective $t_{mp50\%}$ from the paper. This was achieved by proposing a set of criteria that defined when the FD profiles had reached a specific state. Each of these criteria was applied to each profile separately and the results plotted against $t_{mp50\%}$ and tested for correlation.

The remainder of this thesis is presented in eight chapters. Chapter 2 presents an overview of reverberation, including details of RIRs, artificial reverberation, convolution and HAR. Chapter 3 covers fractal geometry and the Higuchi method for estimating FD. Chapter 4 provides an overview of the previous methods for predicting t_{mix} and t_{mp} , as

¹ <http://www.mathworks.com/matlabcentral/fileexchange/30119-complete-higuchi-fractal-dimension-algorithm>

well as details of Lindau study that compared the effectiveness of these predictors. Chapter 5 details the methodology involved in this project and Chapter 6 provides an analysis of the results. Chapter 7 includes the conclusions and future potential work from this project. Chapter 8 outlines the Bibliography. Chapter 9 is the Appendix, which provides all relevant MATLAB code, as well as a more detailed mathematical description of ISM, convolution, and digital filters used in artificial reverberation.

2 OVERVIEW OF REVERBERATION

Reverberation occurs in a confined space after a sound is produced at some point within the space, due to reflections of the sound wave off of surfaces. The energy within the sound wave will be partially reflected depending on the absorption properties of surface material. Provided there is a direct path between the sound source and the listener, after a short initial period of time the listener will hear the direct sound followed by the first reflections from the surfaces. These initial reflections are known as the early reflections and provide the listener with details of the geometry of the room and the location of the source. Eventually the density of the reflections will be such that the listener will no longer be able to distinguish between them. These reflections are known as the late reflections, or the reverberation tail, and provide the listener with a sense of the spaciousness of the room, as well as defining the room's unique timbre or 'sound'. The combination of these early and late reflections is known as *reverberation* and the point at which the early reflections give way to the late reflections is known as the *mixing time*.

The reflection profile of a room is generally represented as a RIR. This RIR shows the variation in sound pressure level (SPL) over time for some point in the enclosed space after a sound has been produced. The amount of reverberation a space will produce is described as the time it takes the SPL to reduce by 60dB (which is equivalent to $1/1,000,000^{\text{th}}$) compared to the original level. It can be calculated from Sabine's reverberation equation [Sabine, 1993], or manually using a loud, short impulse, such as a gunshot, and a recording device. Due to the absorption characteristics of the surface materials, the reverberation time (RT) of an enclosed space will be frequency dependant, so RT is often measured over frequency bands of 1/6, 1/3 or single octaves [Giesbrecht et al, 2009].

2.1 Sabine's Reverberation Equation

In 1898 Wallace Sabine proposed that the time (i.e., RT) required for the SPL of a sound in an enclosed space to reduce by 60dB was proportional to the volume (V) of the space

and inversely proportional to the total absorption (Sa), where the total absorption is the sum of the surface area (A) covered by a particular material multiplied by the materials absorption coefficient (α) [Sabine, 1993].

The Sabine reverberation equation is given by,

$$RT = 0.161 \frac{V}{Sa}, \quad (2.1a)$$

where

$$Sa = \sum_{i=1}^n \alpha_i A_i = \alpha_1 A_1 + \alpha_2 A_2 + \alpha_3 A_3 + \dots, \quad (2.1b)$$

and V is measured in m^3 , A in m^2 , and Sa is measured in *Sabins* ($m^2 \alpha$).

2.2 Room Impulse Response

A room impulse response (RIR) describes the variation in the SPL over time from the instant a sound is produced in an enclosed space from the perspective of a single point in that space. A RIR is divided into three main sections; the direct sound, the early reflections and late reflections (or reverberation tail). A hypothetical RIR profile is demonstrated in Figure 2.2a, showing the SPL variation at one point in space; from the direct sound to the early reflections and finally the indistinguishable late reflections. The characteristics of a RIR are determined by the source and receiver positions, the absorption properties of surface materials, geometry of the room, and the extent of specular diffusion due to uneven surfaces or objects in the room.

There are a number of methods available to procure a RIR of an acoustic space [Stan et al, 2002]. The basic principle of all these methods involves the input of a known and reproducible signal into an acoustic environment and a measurement taken of the output produced by the environment.

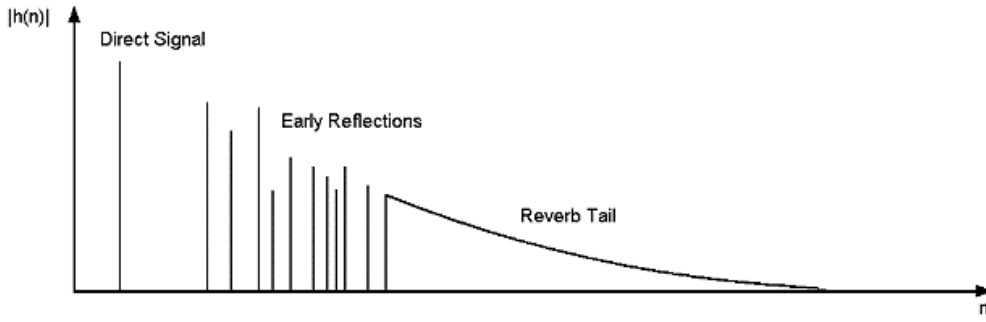


Figure 2.2a. Theoretical RIR showing distinct sections of direct sound, early and late reflections.

A typical real RIR is shown in Figure 2.2b, where an SPL varying about zero point is observed. The RIR of a hypothetical room can be produced by using a model such as the image-source model (ISM) [Allen & Berkley, 1979], which produces an idealised SPL profile with only positive values as seen in Figure 2.2c. A description of ISM is presented in Section 2.4.1 and the mathematical details are presented in Appendix 3.1.

2.2.1 Mixing Time

The point at which the reflections within a room can be defined as the late reflections may be determined by the statistical behaviour of the reflections in relation to each other in the room. Once the reflections have been determined to have statistically lost any memory of their initial state, they are said to be diffuse or mixed. The time from the initial input into the system to this state of statistical independence is defined as the mixing time, t_{mix} .

Another definition for the late reflections is the point at which no extra, perceptual information from reflections can be obtained at one location over any other in the room. This is known as the perceptual mixing time, t_{mp} . This point will occur before t_{mix} due to auditory and cognitive masking effects [Meesawat & Hammershoi, 2003]. There are a number of factors that will influence the time taken to reach t_{mix} or t_{mp} , such as a room's volume, a large variance in surface absorption coefficients (i.e., rooms with glass windows and floor carpeting), and rooms that are coupled or irregular in shape. Rooms where there is a lack of irregular objects to increase diffusion of the reflections will also increase the time taken to reach t_{mix} or t_{mp} . Mixing times will also be frequency dependant due to the absorption characteristics of surface materials [Lindau 2010].

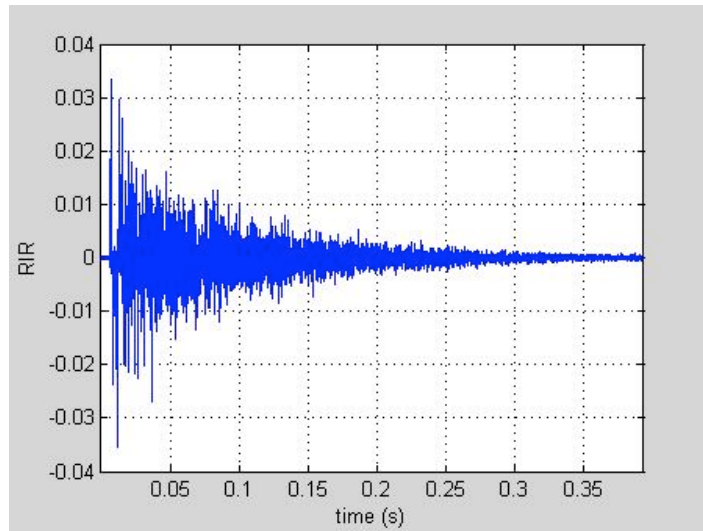


Figure 2.2.b. A typical, real RIR of a room showing SPL variations about zero on the y-axis over time.

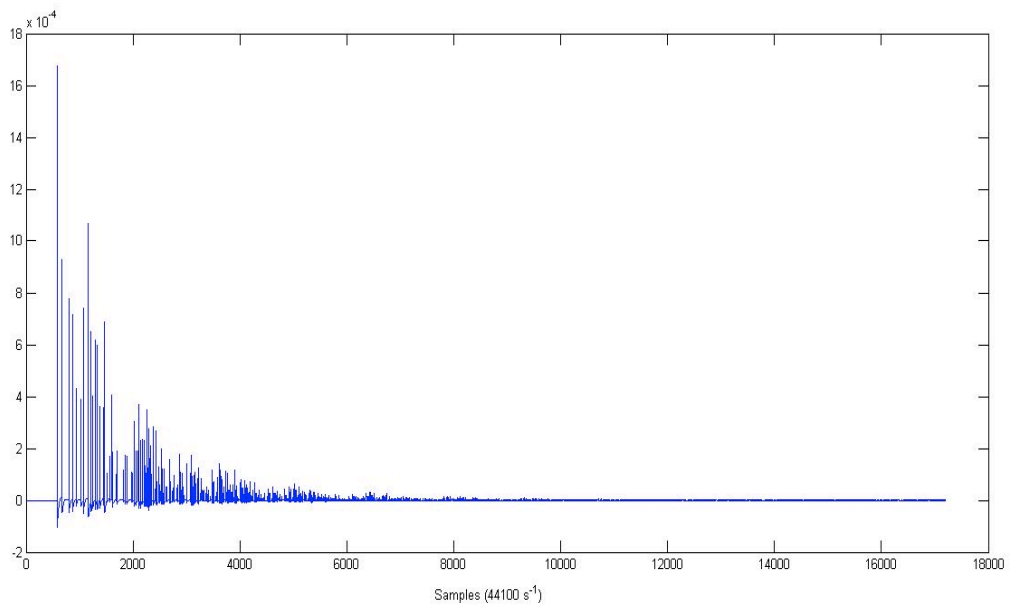


Figure 2.2.c. RIR created using the ISM [Allen & Berkley, 1979] showing relative, positive SPL variation on the y-axis over time.

A number of algorithms have been proposed for calculating t_{mix} from RIRs. These prediction methods are described in detail in Section 4.3 and are based on either how the sound pressure, frequency based energy, or rate of increase in reflections changes over time.

There have been attempts made to derive t_{mp} using various models, and these will be explained in detail in Section 4.2. These attempts look at the physical parameters of the room such as volume, surface area and RT, with consideration paid to the behaviour of the auditory system.

2.2.2 Diffusion of Reflections in a Room

In an ideal reflective room the sound energy will diffuse evenly and quadratically through the room's space over time. This will be met with a quadratic increase in density of reflections [Kuttruff, 2000]. In the ideal reflective room a uniform angular distribution of sound energy flux and a constant acoustical energy will be observed at any point in the room. However, due to the absorption properties of surface materials, there will be a net loss of energy towards the surfaces, biasing the reflection diffusion [Polack, 1992]. Also, due to even slight anomalies on surfaces there will be a tendency towards an ever-increasing level of diffusion due to specular reflections, thus the mixing of reflections in a real room can never be perfectly diffuse or stable.

2.3 Digital Reverberation

Digital reverberation can be achieved by convolving an input signal with a real or simulated RIR, or by processing a signal through a network of digital filters. The method of using a simulated RIR, or processing a signal using filters, is known as artificial digital reverberation. The following sections of this chapter will deal with the fundamentals of digital reverberation.

2.3.1 Convolution

Convolution is a mathematical operation that is performed on two functions to produce a third function. The convolution of two functions f and g is represented as $f * g$ and is defined as '*the integral of the product of the two functions after one has been reversed and shifted*'. If f and g are both complex-valued functions, their convolution can be generally defined as,

$$(f * g)(x) = \int f(y)g(x - y)dy = \int f(x - y)g(y)dy. \quad (2.3.1a)$$

When used in digital reverberation, convolution can be carried out in the time or frequency domain, as convolution in the time domain is equivalent to complex multiplication in the frequency domain. This relationship can be represented as,

$$y(n) = x(n) * h(n) = X(z) \cdot H(z) \quad (2.3.1b)$$

Time domain convolution has the advantage that it can be performed in real time; however it is much more computationally demanding than frequency domain convolution. The disadvantage of using frequency domain convolution however is that the whole length of the input functions must be translated from the time domain and must be present *a priori* before the convolution can occur. To take advantage of the real-time process of time domain convolution and the efficiency of frequency domain convolution, a number of methods, collectively known as block convolution, have been developed² [see Morse & Feshbach, 1953; Hirschman & Widder, 1955; Bracewell 1965; Press et al 1992]. The details of convolution in both the time and frequency domains, as well as a description of block convolution are presented in Appendix 2.

2.4 Artificial Digital Reverberation

The objective of artificial digital reverberation (ADR) is to simulate the reverberation of real acoustic environments and apply this to acoustic recordings in the digital domain. ADR is achieved by either convolving an input signal with a simulated RIR or by processing an input signal using an algorithm comprising a network of digital filters. Each method has its own benefits and drawbacks.

Convolving a signal with a simulated RIR produces a high quality reproduction of a room's characteristics but can be computationally expensive. Convolution also limits certain parameters such as source and receiver positions.

Processing an input signal using digital filters produces a series of delayed and attenuated repetitions of the input signal (in the case of sound this would produce echoes). This method has the advantage of being highly flexible in regards to the type of sound produced and often allows for control of a variety of input parameters [Murphy & Stewart, 2007]. One criticism of this method is its lack of authenticity when compared to the characteristics of real-world acoustic environments [Jot, 1997; Murphy & Stewart, 2007].

² <http://www.ericweisstein.com/encyclopedias/books/Convolution.html>

One option, that attempts to make use of the best aspects of both approaches, is hybrid artificial reverberation (HAR). In HAR, convolution with a simulated RIR is used to recreate the details of the early reflections while a filter-based algorithm is used to approximate late reflections [Murphy & Stewart, 2007; Primavera et al, 2010]. HAR is discussed in more detail in Section 2.5. A more detailed description of ADR can also be found in Appendix 4.

In the following sections of this chapter the ISM for creating simulated RIRs is discussed as well as the basics of convolution and the filter based, algorithmic method for creating digital reverberation.

2.4.1 Image Source Method

The image-source method (ISM) devised by Allen and Berkley [Allen & Berkley, 1979] is a method for generating simulated room impulse responses (RIRs). This method generates an RIR with source and receiver positions by imagining a number of image-sources spaced out on an infinite grid of mirror rooms expanding in all three dimensions. Figure 2.4.1 shows a number of image-sources, I_i , in the x-y plane with the original source, S , and receiver, R , positions. The contribution of each image source to the receiver is a replica of the original source signal delayed by a lag time, τ , and attenuated by an amplitude factor A . The mathematics for this method is described in detail in Appendix 3.1.

2.4.2 Algorithmic Artificial Reverberation

The comb filter and all-pass filter are the basic filters from which complex algorithms have been created in an attempt to reproduce the decay characteristics of different acoustic environments. Both these filters are known as Infinite Impulse Response (IIR) filters. All digital filters are either Finite Impulse Response (FIR) or IIR filters, with IIRs being a more efficient form of filter. An IIR is characterised by always containing a feedback element and its impulse response never completely decaying to zero.

In the following sections each filter will be briefly described, and how they contribute to the modification of a signal to produce an approximation of real acoustic environments is explained. See Appendix 5 for a detailed breakdown of the filters including background on different analysis techniques of digital filter algorithms.

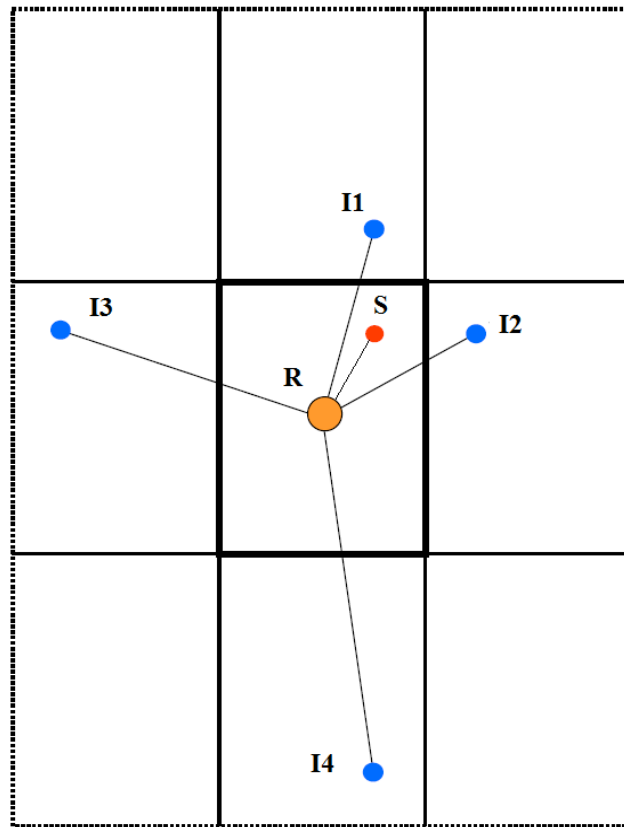


Figure 2.4.1: Image sources in the x-y plane. R represents the receiver position, the circle marked S is the original source with the other circles, and I_i being image-sources in adjacent mirror rooms, where $i = 1, 2, 3, 4$.

2.4.2.1 The Comb Filter

The comb filter (IIR type) comprises a simple delay and feedback line. This produces an exponentially decaying time domain impulse response and is analogous to continued reflections between two facing walls. Figure 2.4.2.1a shows a typical block diagram and impulse response of a comb filter. Further mathematical details of the comb filter can be found in Appendix 5.1.

The term ‘comb filter’ is derived from the topology of the frequency response resulting from the filter’s implementation. A typical comb filter frequency response can be seen in Figure 2.4.2.1b.

2.4.2.2 All-pass Filter

All-pass filters are used to maintain the time-dependence of a reverberation algorithm without influencing its frequency response. The construction of the all-pass is similar to that of a comb filter but achieves frequency independence with the use of a feedforward

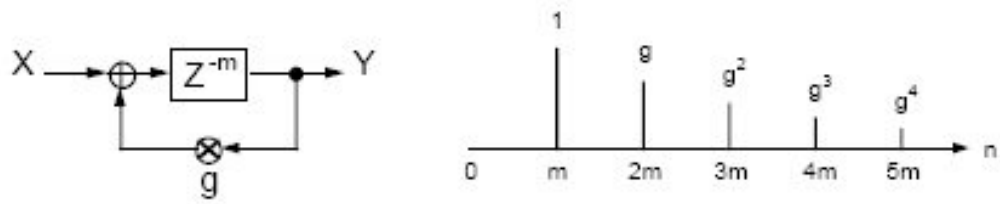


Figure 2.4.2.1a: Block diagram and Impulse response of a comb filter (See Appendix for details).

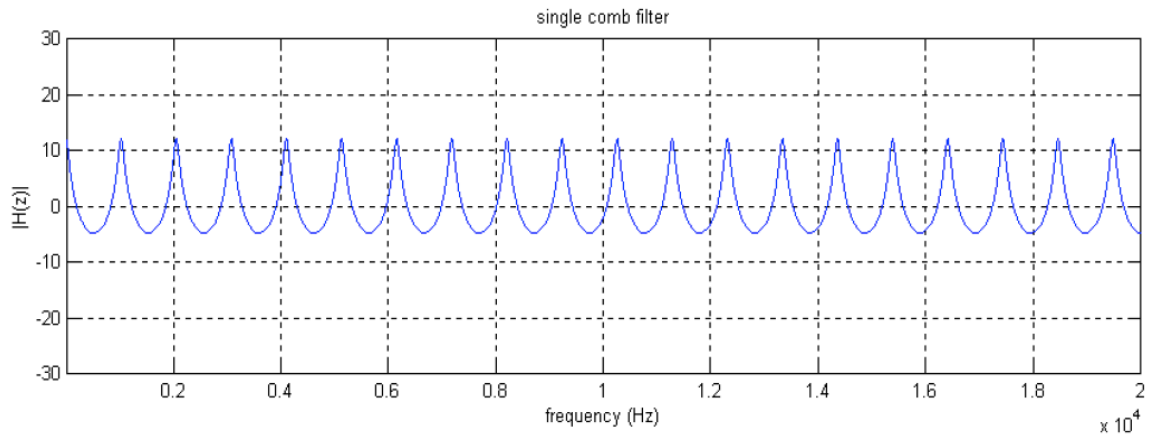


Figure 2.4.2.1b: Frequency response of a comb filter.

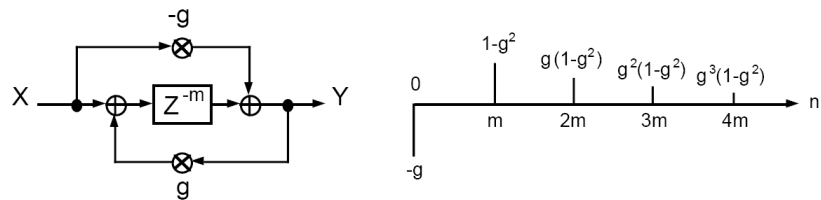


Figure 2.4.2.2a: Block diagram and impulse response of an all-pass filter.

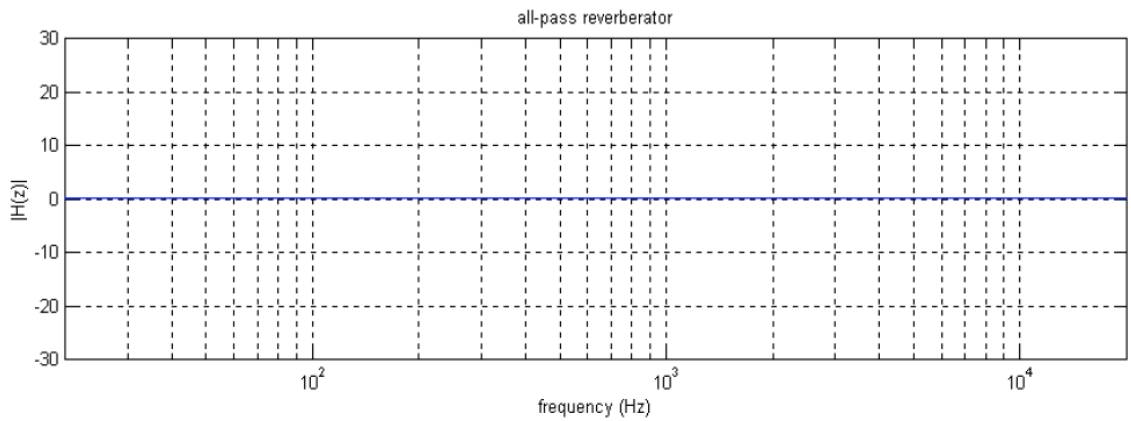


Figure 2.4.2.2b: Frequency response of all-pass filter.

path. See Appendix 5.2 for a more detailed mathematical explanation of frequency independence. Figure 2.4.2.2a shows a block diagram in which the feedforward and feedback elements can be seen; the filter's impulse response is also shown in this figure. Figure 2.4.2.2b demonstrates the flat frequency response of this filter.

The all-pass filter, although producing a flat frequency response overall, will have a non-flat frequency shift due to phase shift, short time intervals, and transient sounds. This gives the all-pass filter a unique and recognisable timbre.

2.4.3 Filter Implementation in Algorithmic Artificial Reverberation

In order to approximate the behaviour of real RIRs, a number of configurations of all-pass and comb filters have been proposed.

In 1962 Manfred Schroeder first proposed two digital artificial reverberator types [Schroeder, 1962]. The first was made up of a series of five all pass filters (see Figure 2.4.3.1). The second comprised a combination of four comb filters in parallel followed by two all-pass filters in series (see Figure 2.4.3.2). Although the first reverberator type has a flat frequency response, it still suffers from transient 'coloration' of the output sound. A further problem with the first reverberator is that its amplitude does not display an exponential decay. The first reverberator type also lacks the build up in reflection density provided by the combination of all-pass and comb filters in the second reverberator. These problems are overcome in the second reverberator type. In this second type, the comb filters job is to recreate reflections between parallel walls and imitate the air absorption of the sound, with their frequency responses modelling the normal modes of the space as seen in a real RIR.

Schroeder's second reverberator type was improved by Moorer by having six comb filters in parallel, each with its own single pole low-pass filter, followed with one all-pass filter. This type can be seen in Figure 2.4.3.3 [Moorer, 1979].

In 1992, Bill Gardner proposed three reverberators of similar structure, specific to a room's approximate size; small, medium or large [Gardner, 1992]. See Figure 2.4.3.4 for a graphic representation of these three room sizes, which includes specific variable

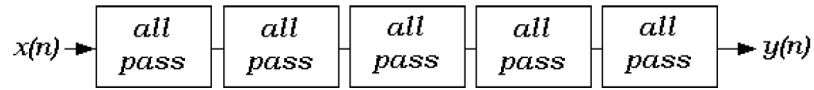


Figure 2.4.3.1: Schroeder Reverberator 1.

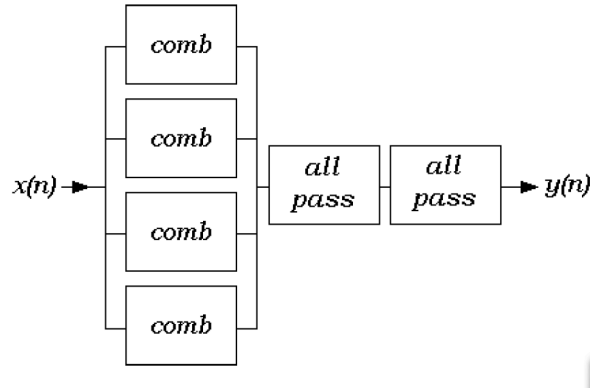


Figure 2.4.3.2: Schroeder Reverberator 2.

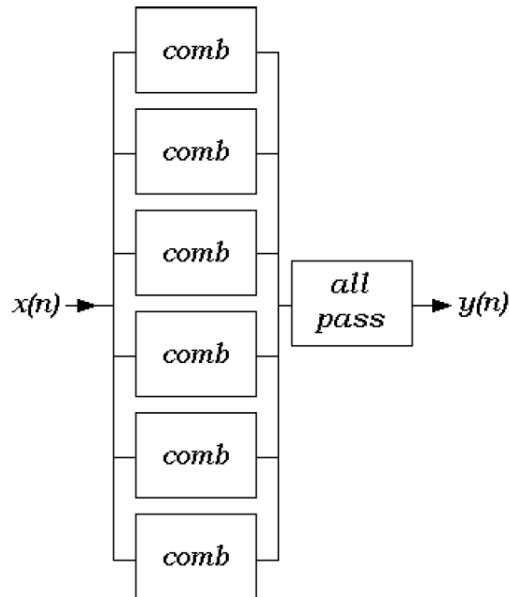
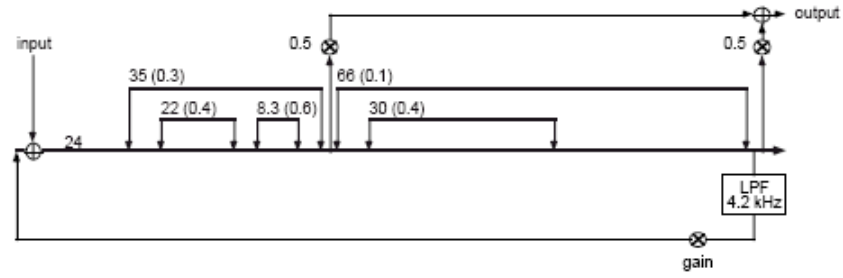


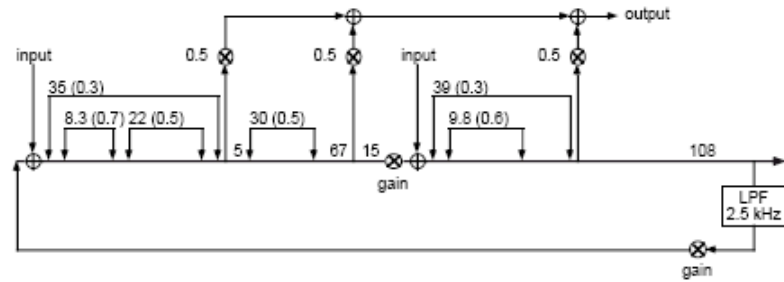
Figure 2.4.3.3: Moorer reverberators.

values. Gardner defines a small room as having a RT of 380 - 570ms, a medium room as having a RT of 580 - 1290ms and rooms with a RT over 1300ms as large. His reverberators comprise a single delay line looped on itself through a variable gain value with nested all-pass filters and delays, followed by a single pole low pass filter. The output signal is generated by summing the output from various points along the delay line. The basic structure of Gardner's reverberator can be seen in Figure 2.4.3.5 (for a large room). This particular reverberation algorithm is appropriate for creating late reflections only and is often used in HAR.

Small room reverberator:



Medium room reverberator:



Large room reverberator:

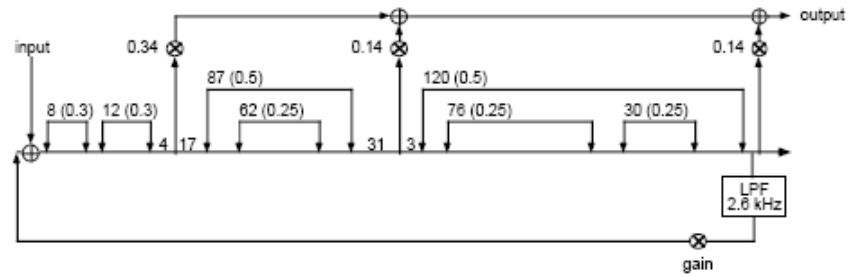


Figure 2.4.3.4: Graphic representation of Gardiner's late reflections artificial digital reverberations algorithms for small, medium and large rooms.

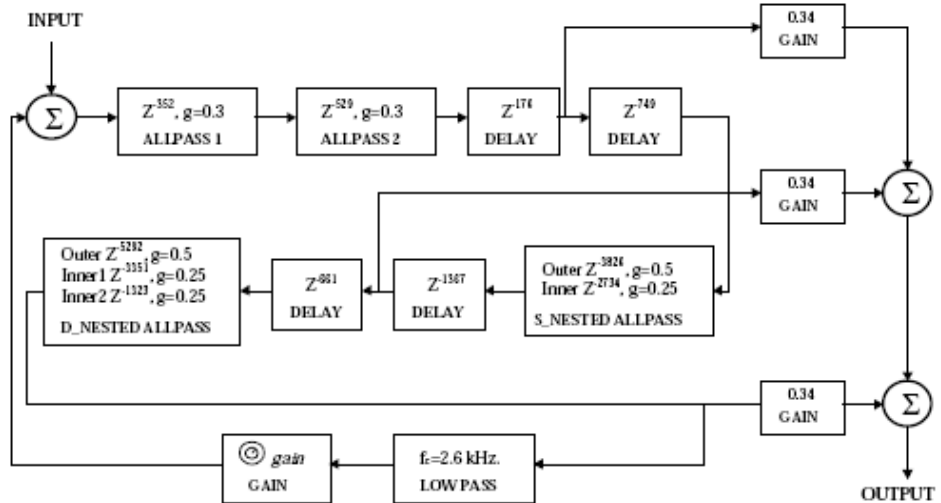


Figure 2.4.3.5: Block Diagram of Gardiner's reverberators for a large room.

2.4.4 Feedback Delay Networks

Feedback delay networks (FDN) were first suggested for use with ADR algorithms by Michael Gerzon after observing that individual feedback comb filters yielded poor quality sound, whereas when several were cross-coupled the result was of superior quality [Gerzon, 1971; Gerzon, 1972]. An FDN is a feedback comb filter with N feedback channels where the delay line and feedback gain elements are replaced with matrices. The delay line lengths are kept mutually prime to maximise the pseudo-randomness of the reverberators. See Appendix 5.3 for a mathematical description of FDN. Figure 2.4.4.1 shows a FDN network with three feedback channels that are cross-coupled in a mixing matrix, Q , described as,

$$Q = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix} . \quad (2.4.4a)$$

Jot used this concept to create an ADR algorithm that has individual RTs for separate frequency bands, promoting the absence of tonal coloration in the late reflections, as well as being able to be specified in terms of desired reverberation and frequency response. See Figure 2.4.4.2 for an example of the Jot reverberator [Jot & Chaigne, 1991; Jot, 1992].

Jot added a tonal correction filter, $E(z)$, after the FDN output (see Appendix 5.3 for details) which serves to equalize modal energy amongst the three frequency bands. Figure 2.4.4.2 shows a block diagram of Jot's reverberator which employs an FDN with a tonal correction filter. The output after the correction filter is summed with the input signal, $x(n)$, to produce the final, output signal, $y(n)$.

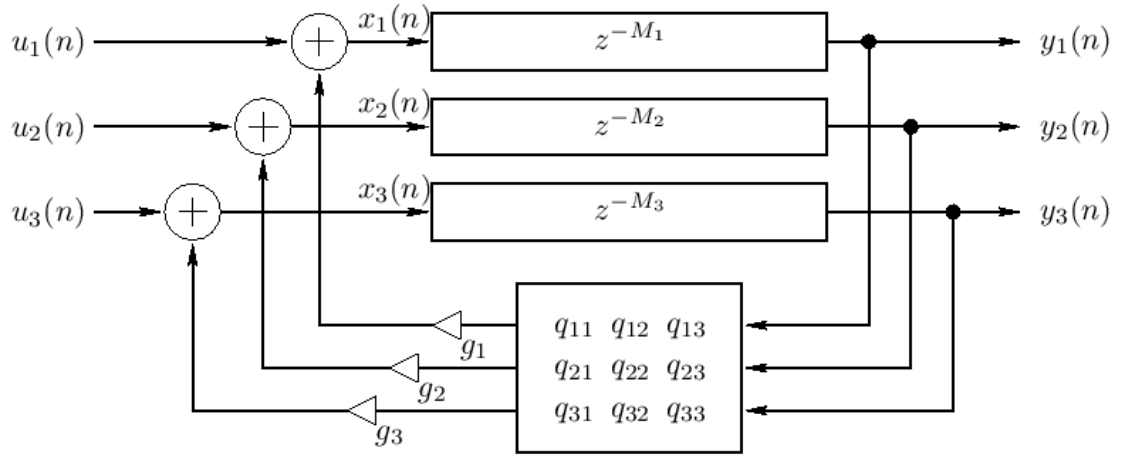


Figure 2.4.4.1: FDN of order $N=3$.

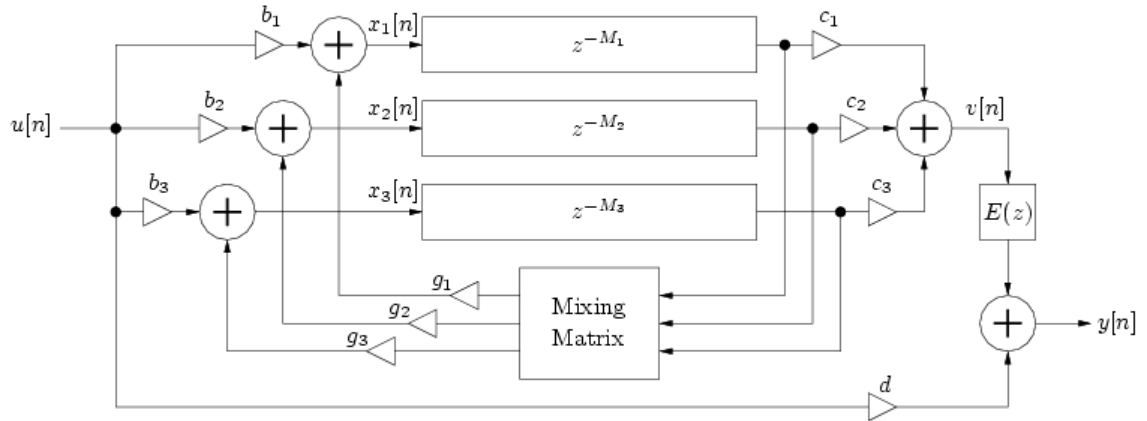


Figure 2.4.4.2: A block diagram of Jot's reverberator employing a FDN with a tonal correction filter.

2.5 Hybrid Artificial Reverberation

Hybrid Artificial Reverberation (HAR) hopes to take advantage of the perceptual acoustic accuracy of using convolution with a simulated RIR, and the flexibility and efficiency of algorithmic reverberation to produce computationally efficient and high fidelity digital reverberation. HAR uses a convolution of a RIR with an input signal, truncated at t_{mix} (or ideally t_{mp}), to generate the early reflections. This is then added to algorithmically generated late reflections. Often the impulse will be analysed to determine a number of parameters for the late reflection, such as the delay and gain factors [Meesawat & Hammershoi, 2003; Murphy & Stewart, 2007]. The location of t_{mix} (or t_{mp}) is of great importance when designing for HAR as the computational cost of convolution must be

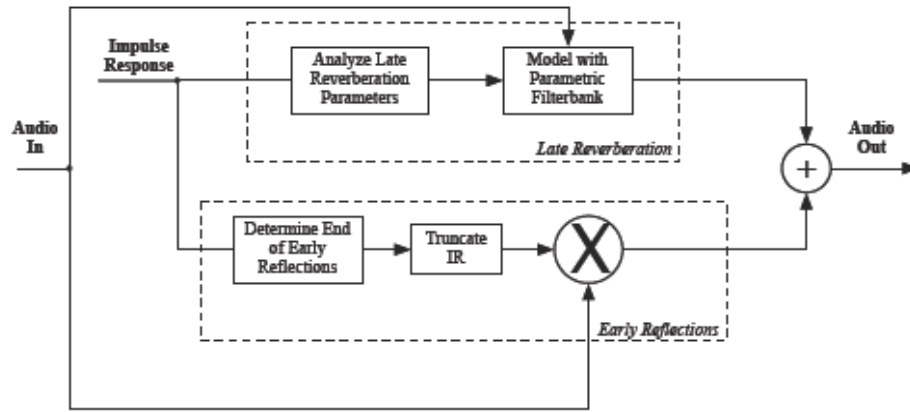


Figure 2.5: Structure used in HAR [Murphy & Stewart, 2007].

kept to a minimum, but equally as important is ensuring that ADR does not begin too soon, introducing audible artefacts.

In order to find t_{mix} of the input signal, some form of analysis must be carried out at the start of the HAR [Lindau, 2010]. Figure 2.5 above shows the structure for a HAR proposed by Murphy & Stewart [2007] showing an RIR, truncated at a predefined point, being convolved with an input signal for early reflections and a reverberation algorithm for the late reflections. These two separate signals are summed before output.

3 FRACTAL GEOMETRY

Fractal Geometry is a modern addition to the mathematician’s toolbox that has allowed the accurate description of a broad range of natural processes. This new mathematical vocabulary has facilitated technical advances and new levels of academic understanding in fields ranging from telecommunications to human-integrated technology.

Benoit B. Mandelbrot first coined the term ‘fractal’ in 1982 to describe “*a rough or fragmented geometric shape that can be split into parts, each of which is (at least approximately) a reduced-size copy of the whole*” [Mandelbrot, 1982]. The two defining aspects of a fractal are that it exhibits self-similarity at arbitrary levels of magnification, and that it can be described by a non-integer dimension whose value is greater than that of its topological dimension. Mathematically, a fractal can be generated by repeated implementation (iteration) of an equation, which by its nature will result in recursive self-similarity.

3.1 Development of Fractal Geometry

The idea of self-similarity was first proposed in the 17th century by Gottfried Leibnitz’ attempts to define the properties of a straight line. However, the basis of the mathematics of fractal geometry dates back to 1872 with Karl Weierstrauss’ demonstration that a function could be both continuous everywhere, but nowhere differentiable. Consider the *Weierstrauss function*,

$$w(t) = \sum_{k=0}^{\infty} \gamma^k \cos(\beta^k t). \quad (3.1a)$$

With scaling in the time domain t , by a factor β , and for $\gamma = 1$, Weierstrauss was able to show that

$$w(\beta t) = \sum_{k=0}^M \cos(\beta^{k+1} t) = \sum_{k=1}^{M+1} \cos(\beta^k t). \quad (3.1b)$$

This magnification-independent self-similarity can be demonstrated acoustically by choosing a β factor of 2 and keeping $\gamma = 1$. This function will produce a summed waveform, which will sound the same to the human ear whether played at half or twice the speed. Similarly, by choosing a β factor of $^{12}/_{13}$, upon doubling the speed of play, a perceived decrease of a semitone is heard [Schroeder, 1989].

Weierstrauss' mathematical description was followed up by a more graphical description by Helge Von Koch in 1904 with the Koch Curve. The curve is constructed following a set of rules that are repeated *ad infinitum*. First a line segment is divided into three segments of equal length. An equilateral triangle is then constructed that has the middle segment from the original line segment as its base. Finally the line segment that forms the base of the triangle is removed. See Figure 3.1a for a depiction of Koch iterations [Mandelbrot, 1982; Schroeder, 1989].

Following the work of Weierstrauss and Koch, investigations of iterated functions in the complex plane were carried out by Henri Poincaré, Felix Klein, Pierre Fatou and Gaston Julia (responsible for the Julia set) [Mandelbrot, 1982; Schroeder, 1989], who described a series of new fractal systems. Without the means to visualise this new set of unusual functions, mathematicians were unable to ascribe them any significance beyond mere curiosity.

With the advent of modern computer technology, fractal geometry was released from the limitations of manual calculation, allowing the visualisation of rapid and numerous iterative calculations in the real and complex planes. By being able to visualise the results and evolution of millions of iterations of these fractal functions, the true importance of these curious functions and their uncanny resemblance to natural processes could finally be realised.

In the 1960's, Benoit B. Mandelbrot started investigating self-similarity, publishing '*How Long is the Cost of Britain? Statistical Self-Similarity and Fractal Dimension*' [Mandelbrot, 1967]. Using computers to run large numbers of iterations on the different

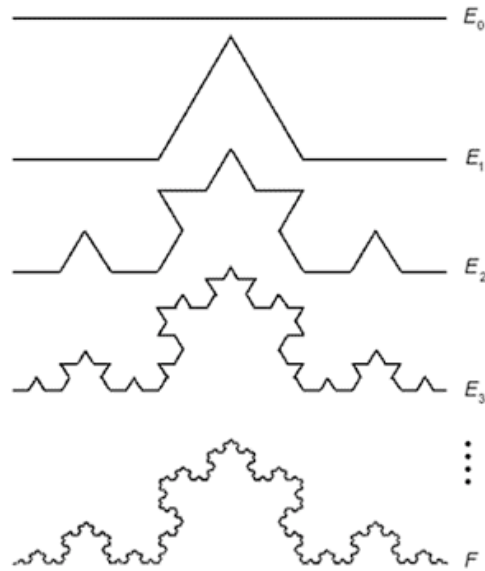


Figure 3.1a – Iterations of the Koch Curve.

fractal functions discovered by Poincaré, Klein, Fatou and Julia [Mandelbrot, 1982] he was able to visualise their intricate self-similarity for the first time. In his work, ‘*The Fractal Geometry of Nature*’ [Mandelbrot 1982], Mandelbrot presented a visual rendering of the fractal functions with a discussion on the resemblance of their structure to that of structures found in nature. In this work, Mandelbrot emphasised the potential use of fractals for describing and modelling real-world phenomena, from the coastlines of countries to biological processes. Mandelbrot’s observations opened the world of fractal geometry beyond the often impenetrable domain of mathematics, inspiring a wide range of researchers and technicians. This has resulted in fractal geometry influencing nearly every field of academia today. The most widely recognised fractal today is undoubtedly the Mandelbrot Set, which was developed by the mathematicians Adrien Douady and John H. Hubbard who named the set in Mandelbrot’s honour [Douady & Hubbard, 1984; Douady & Hubbard, 1985]. This set is reproduced in Figure 3.1b.

3.1.1 Fractal Dimension

The dimension of a fractal is a measure of its complexity and denotes how completely the fractal fills a space. The FD is always larger than the shape’s topological dimension by some amount less than one, and the closer this amount is to one the greater that fractal fills space. The dimension of a fractal is a somewhat abstract concept and a number of viewpoints exist from which to calculate it, each one tackling the problem of how a

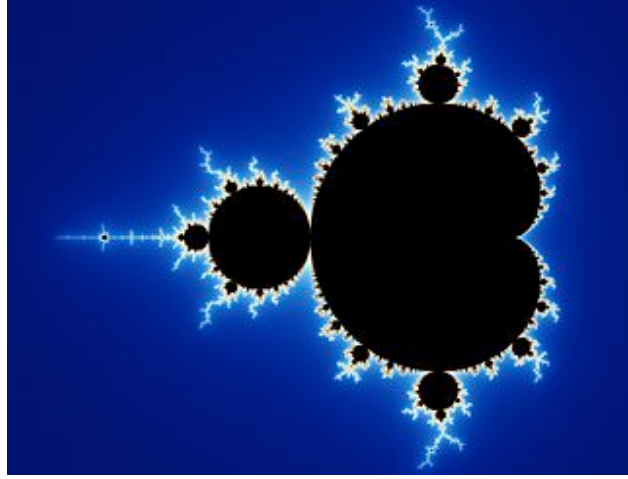


Figure 3.1b – Visualisation of the Mandelbrot Set.

fractal fills space from slightly different perspectives. The two most widely quoted fractal dimensions are the Hausdorff and Box-Counting dimensions. These measures of fractal dimensions, along with most other measures of FD, will nearly always produce the same value, with the exceptions existing below the threshold of anxiety for most common situations (including this study).

The Box-Counting FD is most commonly employed due to its simplicity and is worth examining in detail to get a clearer understanding of the basic principle of FD. To calculate the Box-Counting FD, first imagine that the fractal is lying on a grid of squares. It will take a certain number of these squares to completely cover the fractal. Now imagine systematically reducing the size of the squares over and over. The Box-Counting FD is derived from the number of squares required to cover the fractal changes as the squares get smaller. An example of this method can be seen in Figure 3.1.1, employing the box-counting method to calculate the FD of the coastline of the United Kingdom.

If we have a number of boxes, $N(\epsilon)$, of side length ϵ , required to cover a fractals, S , then the Box-Counting FD is defined as,

$$FD_{box}(S) := \lim_{\epsilon \rightarrow 0} \frac{\log N(\epsilon)}{\log(1/\epsilon)} \quad , \quad (3.1.1a)$$

where the logarithm can be to any base.

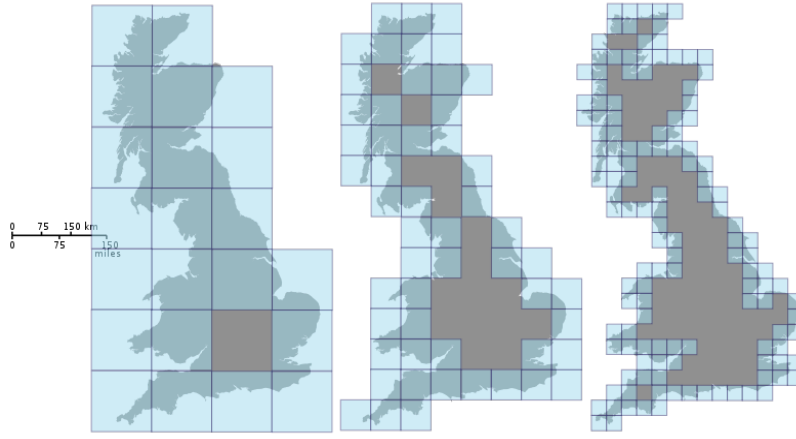


Figure 3.1.1. Box-Counting FD method, employed on the coast of the UK.

3.1.2 Calculating the Fractal Dimension of Real-World Fractal Systems

For real-world fractal systems, the FD must be estimated, as invariably the system will not be self-similar at all levels of magnification. The Higuchi Algorithm was used for estimating the FD in this investigation [Higuchi, 1988]. One useful property of FD algorithms is that they will give a value for the FD of any space-filling shape, whether it is self-similar or not. This can be useful, as it will still give an indication of the shape's complexity. After comparison to other algorithms, the Higuchi algorithm was chosen due to its accuracy for estimation against fractals of known FD (see Figure 3.1.2a), and its ability to cope with small data sets (see Figure 3.1.2b) [Higuchi, 1988; Raghavendra & Dutt, 2010].

The Higuchi Algorithm estimates the FD of a waveform using the following method; first, a portion of the waveform is represented by $y(1), y(2), \dots, y(N)$, where N is the total number of samples in the portion. From a given sample portion, k , new sub-portions are constructed and represented by y_m^k , defined as,

$$y_m^k = \{y(m), y(m+k), y(m+2k), \dots, y(m+Mk)\}, m = 1, 2, \dots, k \quad (3.1.2a)$$

where m and k are integers, indicating initial time and interval time respectively, and $M = \lfloor (N - m) / k \rfloor$, where $\lfloor \cdot \rfloor$ denotes the magnitude. For each of the sub portions, y_m^k , constructed, the average length $L_m(k)$ is computed as

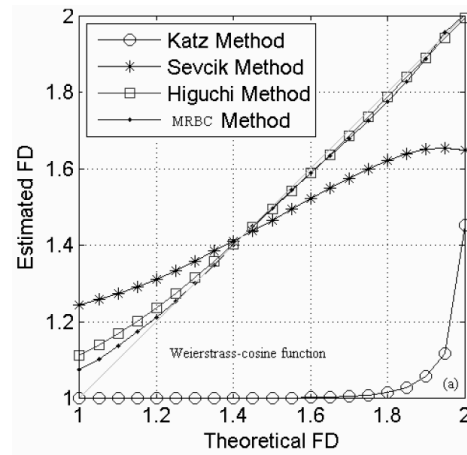


Figure 3.1.2a: Estimated FD against Theoretical FD for a number of FD algorithms.

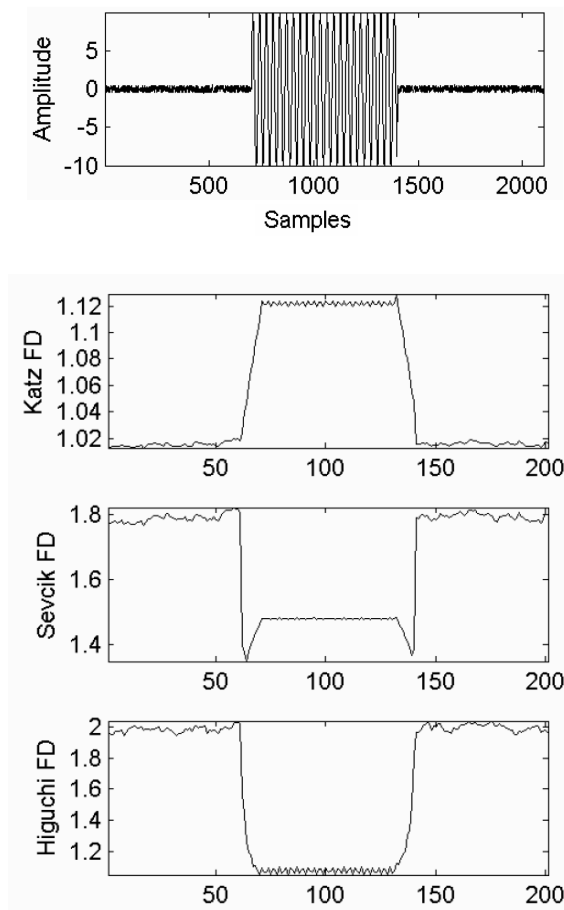


Figure 3.1.2b: Cascade showing effect on FD for various algorithms, going from a random to sinusoid waveform over a short sample length. Note the Higuchi algorithm swiftly moving to a FD of 1 with little noise upon introduction of sinusoidal waveform.

$$L_m(k) = \frac{1}{k} \left\{ \frac{N-1}{Mk} \sum_{i=1}^M (|y(m+ik) - y(m+(i-1)k)|) \right\} \quad (3.1.2b)$$

where $(N-1)/Mk$ is a normalization factor. The length of the portion $L(k)$ for the time interval k is computed as the mean of the k values, for $m=1,2,\dots,k$, where $L(k) = \sum_{m=1}^M L_m(k)$. If $L(k)$ is proportional to k^{-D} , the curve describing the shape of the portion is fractal-like with the dimension D . Thus, if $L(k)$ is plotted against $k=1,\dots, k_{\max}$, on a double logarithmic scale, the points should fall on a straight line with a slope equal to $-D$. The least-square linear best fitting procedure is applied to the graph of $\ln(L(k))$ vs. $\ln(1/k)$. The coefficient of linear regression of the plot of $\ln(L(k))$ versus $\ln(1/k)$ is taken as an estimate of the fractal dimension of the portion. The value of interval time used is taken as $k = 1, 2, 3, 4$, with $k = [2(j-1)/4]$ for k larger than 4, where $j = 11, 12, 13, \dots$ and $\lfloor \cdot \rfloor$ denotes Gaussian notation in Equation 3.1.2b.

4 PREDICTING THE MIXING TIME

The methods for predicting the transition from early reflections to late reflections in a reverberant system fall into two categories. The first looks at the statistical behaviour of the reflections in the system by considering the behaviour of the room's RIR. These methods employ mathematical algorithms to produce the mixing time (t_{mix}). The second category of predictors considers how the human auditory system perceives information from reflections in an acoustic environment, and attempts to derive the perceptual mixing time (t_{mp}) from a model that considers this factor and the characteristics of the environment.

4.1 Model Predictors

The reflection density of a rectangular room at any time, t , is described as

$$\frac{dN}{dt} = \frac{4\pi \cdot c_0^3 \cdot t^2}{V} \quad , \quad (4.1a)$$

where c_0 is the speed of sound (ms^{-1}) and V the room volume (m^3). Values of $1000 s^{-1}$ [Schroeder, 1962] $4000 s^{-1}$ [Rubak & Johansen, 1999] and $10000 s^{-1}$ [Griesinger, 1989] have been suggested as the threshold beyond which the human ear can no longer derive information from distinct reflections, thus setting the lower limit of t_{mp} .

Through experiment, Schreiber [1960] determined a just distinguishable reflection density of $2000s^{-1}$. Solving this for t in Equation 4.1a gives

$$t_{mp} = 2\sqrt{V} \text{ (ms)}. \quad (4.1b)$$

Polack [1960] has suggested a just distinguishable reflection density of ten reflections within a specific period of 24ms^{-1} , which would equate to a reflection density of 400s^{-1} . This equates to

$$t_{mp} \cong \sqrt{V} \text{ (ms)}. \quad (4.1c)$$

Ruback and Johansen [1999] approached a model-based prediction for t_{mp} based on the mean free path, l_m , of the room. They proposed that once every sound wave had undergone a total of four reflections then they would have statistically lost memory of their initial state, and then the sound field could be considered diffuse. The equation for l_m is written as $l_m = 4 \frac{V}{S} \text{ (ms)}$, where S is the total surface area of the room (m^2). Thus the t_{mp} equation reads

$$t_{mp} = 4l_m \frac{10^3}{c_0} = 4 \cdot \left(\frac{4V}{S} \right) \cdot \frac{10^3}{c_0} = 47 \cdot \frac{V}{S} \text{ (ms)}. \quad (4.1d)$$

From a study of 59 RIRs in concert halls, Hidaka et al [2007] derived a linear regression formula for t_{mix} based on the reverb time at specific frequency bands. For the 500Hz frequency, Hidaka et al. found that

$$t_{mix_{500\text{Hz}}} = 80 \cdot RT_{500\text{Hz}} \text{ (ms)}. \quad (4.1e)$$

4.2 Empirically Derived Predictors

A number of algorithms have been devised to calculate t_{mix} based on experiments looking at RIRs. These will be explained in detail in this section.

The first of the predictors to be looked at was put forward by Abel & Huang [2006], and makes the assumption that the sound pressure amplitudes in a reverberation field takes on a Gaussian distribution. An ‘echo density profile’ is calculated to determine the t_{mix} . For a sliding rectangular window of 500 - 2000 samples, the empirical standard deviation of the sound pressure amplitudes is calculated for each sample index. To determine how

well the empirical amplitude distribution approximates a Gaussian behaviour, the proportion of samples outside the empirical standard deviation is determined and compared to the proportion that is expected for a Gaussian distribution. With increasing time and diffusion, this echo density profile should increase until it reaches complete diffusion (i.e., the moment the late reverberation begins) and records a value of one.

This method has been further refined so that the criterion now waits for the comparison to a Gaussian distribution to reach $1-\sigma_{late}$ (where σ_{late} is the standard deviation of the late echo density).

A second method was proposed by Stewart & Sandler [2007], which involves measuring the kurtosis of the sound pressure amplitudes and comparing this value to that expected for a Gaussian distribution. As a second order cumulant, kurtosis (γ_4) is a measure of the deviation from Gaussian behaviour of a signal. Kurtosis in the normalised form, γ_{4n} , is given by,

$$\gamma_{4n} = \frac{E(x - \mu)^4}{\sigma^4} - 3, \quad (4.2a)$$

where $E(\cdot)$ is the expectation operator, μ is the mean, and σ is the standard deviation of the process. For increasingly Gaussian processes, the normalised kurtosis approaches zero. t_{mix} is derived from the time when γ_{4n} (normalised to a maximum value of one) has reached zero for the first time.

A third approach was proposed by Hidaka et al [2007] that considers t_{mix} in the frequency domain, and aims to determine when an RIR indicates the sound field has become diffuse. To do this, the time-frequency energy distribution of the RIR, $p(t)$, is calculated according to:

$$E(t, \omega) = \left| \int_t^\infty p(t) e^{j\omega t} dt \right|^2. \quad (4.2b)$$

The energy distribution $E(t, \omega)$ is calculated for each RIR beginning with the direct sound, with initial delays having been removed in advance. With increasing time t , $E(t, \omega)$ will continuously contain less early reflections and increasingly more of the statistically

random late reflections. As a second step, Pearson’s product-moment correlation³, $r(t)$, is calculated as a continuous function of time for $E(0:\infty, \Delta\omega)$ and $E(t:\infty, \Delta\omega)$ in arbitrary frequency bands. This will correlate the similarity between a) the energy decay process including the initial state and b) the energy decay process beginning from any time t afterwards in one particular frequency band. Hidaka et al [2007] defines t_{mix} as the instant when $r(t) = 0.367$.

4.3 Perceptual Evaluation of Mixing Time Predictors

In 2010 Alexander Lindau (et al.) won the Audio Engineering Society (AES) Student Technical Paper Award at the 128th AES convention where he presented a peer reviewed paper entitled ‘*Perceptual Evaluation of Physical Predictors of the Mixing Time in Binaural Room Impulse Responses*’ [Lindau et al, 2010]. Lindau et al. presented a comparative study of the perceptual mixing time (t_{mp}) against the various predictors for t_{mix} and t_{mp} (as presented in the previous section of this thesis). The following sections will present the details of this study, the results of which will be used to generate results for this study in Chapter 5 and analysed in Chapter 6.

4.3.1 Method

Binaural RIRs were taken for nine rooms and listening tests carried out to determine the t_{mp} , as judged by 24 participants. In the following sections, details of the method and results of Lindau’s study are presented.

4.3.1.1 Room Selection

The nine rooms used in the study were selected to be representative of the typical range of musical performance venues, and were chosen to be of regular ‘shoebox’ shape to minimise effects from spectral diffusion off irregular surfaces and to ensure long unobstructed path lengths (factors which have been suspected to be critical to mixing theory, as detailed in Chapter 2). Rooms were chosen to obtain incremental variation in volume, V , and average absorption coefficient, α_{avg} (see Table 4.3.1.1). Rooms 1 – 3 were

³ http://en.wikipedia.org/wiki/Pearson_product-moment_correlation_coefficient

	small V	medium V	large V	α_{avg} (RT)
large α (RT)	room 1 216 m ³ α : 0.36 (0.39 s)	room 4 3300 m ³ α : 0.28 (1.15 s)	room 7 8298 m ³ α : 0.33 (1.52 s)	0.32 (1 s)
medium α (RT)	room 2 224 m ³ α : 0.26 (0.62 s)	room 5 5179 m ³ α : 0.23 (1.67 s)	room 8 8500 m ³ α : 0.23 (2.08 s)	0.24 (1.45 s)
small α (RT)	room 3 182 m ³ α : 0.17 (0.79 s)	room 6 3647 m ³ α : 0.2 (1.83 s)	room 9 7417 m ³ α : 0.23 (2.36 s)	0.2 (1.66 s)
avg. Vol.	207 m³	4042 m³	8072 m³	

Table 4.3.1.1: Volume, RT and α_{ave} for nine rooms in Lindau et al 2010 study.

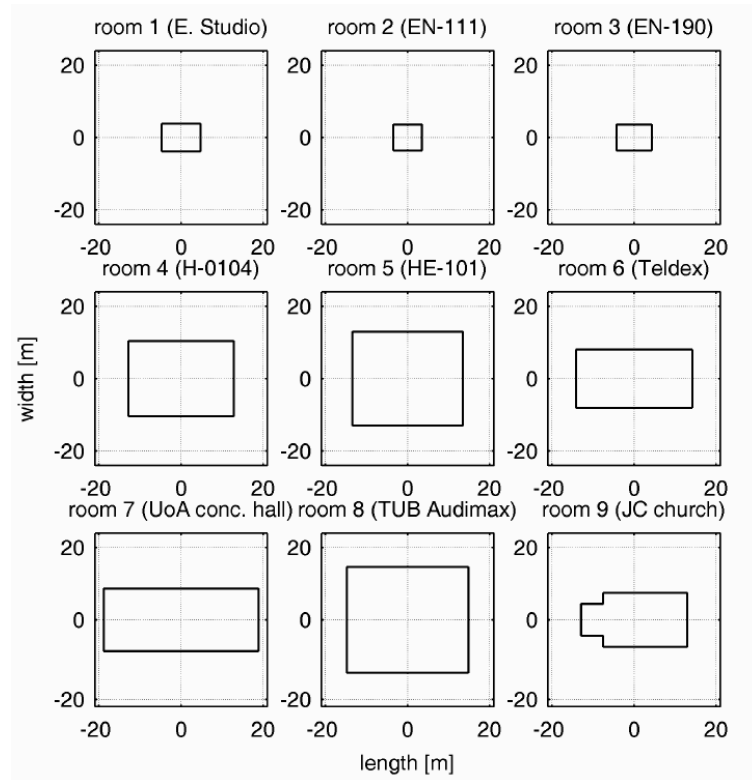


Figure 4.3.1.1: Floor plans for nine rooms from Lindau et al study.

small lecture theatres with chairs and tables, and rooms 4 – 9 were larger rooms with extended audience seating. Figure 4.3.1.1 shows the layouts of these rooms in more detail. The rooms were measured using a laser distance meter to calculate volume.

Binaural room impulse response (BRIR) measurements were made of each room using the automatic head and torso simulator, FABIAN [Lindau, 2006]. For the source sound a 3-way dodecahedron loudspeaker was placed on a stage at one end of each room. The BRIRs were collected for horizontal head orientations within $\pm 80^\circ$ in angular steps of 1° .

4.3.1.2 Listening Tests

Perceptual mixing times were determined through a listening test procedure where the subjects had to discriminate manipulated dynamic binaural simulations from the original ones. The original simulations comprised of the BRIRs, which were updated in real time to account for head movement. In the manipulated simulations, only the early part of the BRIRs corresponded to the subjects' true head position. The convolution results of the late reverberation tail – taken from the BRIR corresponding to the frontal head position – were concatenated to the early output but not changed otherwise. In the tests, the early and late BRIR parts were concatenated at arbitrary instants of 5.8ms (rooms 1 – 3) and 11.6ms (rooms 4 – 9) respectively. Cross-fades of 100ms were used for rooms 1 – 3 and 200ms for rooms 4 – 9 respectively. The stimulus used for the tests was a drum set sample of length 2.5 seconds without reverb tail. A total of 24 subjects of average age were subjected to the nine rooms in randomised order with 20 trials for each room taken.

4.3.2 Results

Due to the poor discrimination between the modified and unmodified BRIRs demonstrated by some of the test subjects, only 10 sets of results were chosen for analysis. The results for the mean perceptual mixing time ($t_{mp50\%}$) from the ten qualifying participants for the nine rooms is shown in Figure 4.3.2, with $t_{mp50\%}$ reflected in the centre of the black line, and 95% confidence intervals (CI) represented by the limits of the line.

4.3.2.1 Model Predictor Results

As Equations 4.1b - 4.1e are solely in terms of volume, V , surface area, S , and reverb time, RT , they can be generalised. The equation that concerns reflection density can be rewritten

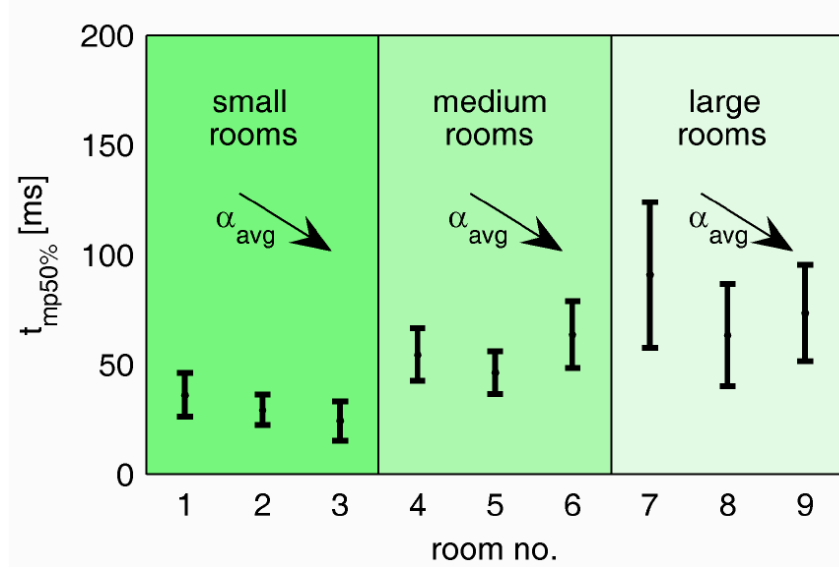


Figure 4.3.2: Mean perceptual mixing time, $t_{mp50\%}$, for the nine rooms with 95% confidence intervals.

$$t_{mp} = k_{ref} \cdot \sqrt{V}, \quad (4.3.2.1a)$$

the equation for the mean free path prediction model becomes

$$t_{mp} = k_{path} \cdot \frac{V}{S}, \quad (4.3.2.1b)$$

and for the reverb time (RT) based predictor, the equation becomes

$$t_{mp} = k_{reverb} \cdot RT_i. \quad (4.3.2.1c)$$

Lindau tested for correlation between $t_{mp50\%}$ and these model predictors as well as against volume (V), surface area (S), and average absorption (α_{ave}) of the rooms. The best model predictor for $t_{mp50\%}$ was the mean free path predictor, (Equation 4.3.2.1b) with an explained variance, R^2 , of 81.5% with a linear regression equation of the form

$$t_{mp50\%} = 20.08 \cdot V/S + 12 \text{ (ms)}. \quad (4.3.2.1d)$$

This was followed by the reflection density model (Equation 4.3.1a) with an R^2 of 78.6%. Volume alone achieved a R^2 of 77.4%. The RT based model (Equation 4.3.2.1c) proved to be a poor predictor with an R^2 of only 53.4%. The average absorption coefficient, α_{ave} , showed no correlation with Lindau's results for $t_{mp50\%}$ with an explained variance of $R^2 = 0.8\%$.

Lindau followed these results up with a correlation analysis against the models and variables for the 95%-percentiles of the distribution for the results for t_{mp} ($t_{mp95\%}$). For $t_{mp95\%}$ the best prediction was found for volume, with an R^2 of 78.7%. All results for $t_{mp50\%}$ and $t_{mp95\%}$, with 95% Confidence Intervals (CI) can be seen in Figure 4.3.2.1.

4.3.2.2 Empirically Derived Predictors

Of the empirically derived predictors, the algorithm offered by Abel & Huang [2006] (criterion I: echo density profile = 1) showed the greatest correlation with $t_{mp50\%}$ with an R^2 of 74.7%. This was followed by the Hidaka et al. [2007] method at 1kHz and 500kHz octave bands, with an R^2 of 57.3% and 56.5% respectively. Abel & Huang's algorithm (Criterion II: Echo density profile = $1 - \sigma_{late}$) produced a R^2 of 50.7% and Stewart and Sandler [2007] came in worst with a R^2 of 37.6%. The Defrance et al [2009] algorithm had to be disqualified from the results as a correlation could only be achieved with the removal of one anomalous result. The results for $t_{mix50\%}$ and $t_{mix95\%}$ with 95% CI are presented in Figure 4.3.2.2.

A summary of the results from Lindau's study is presented in Table 4.3.2.2, which shows that the model predictor based on the mean free path within the room (Equation 4.3.2.1a) proved best at predicting the mean perceptual mixing time.

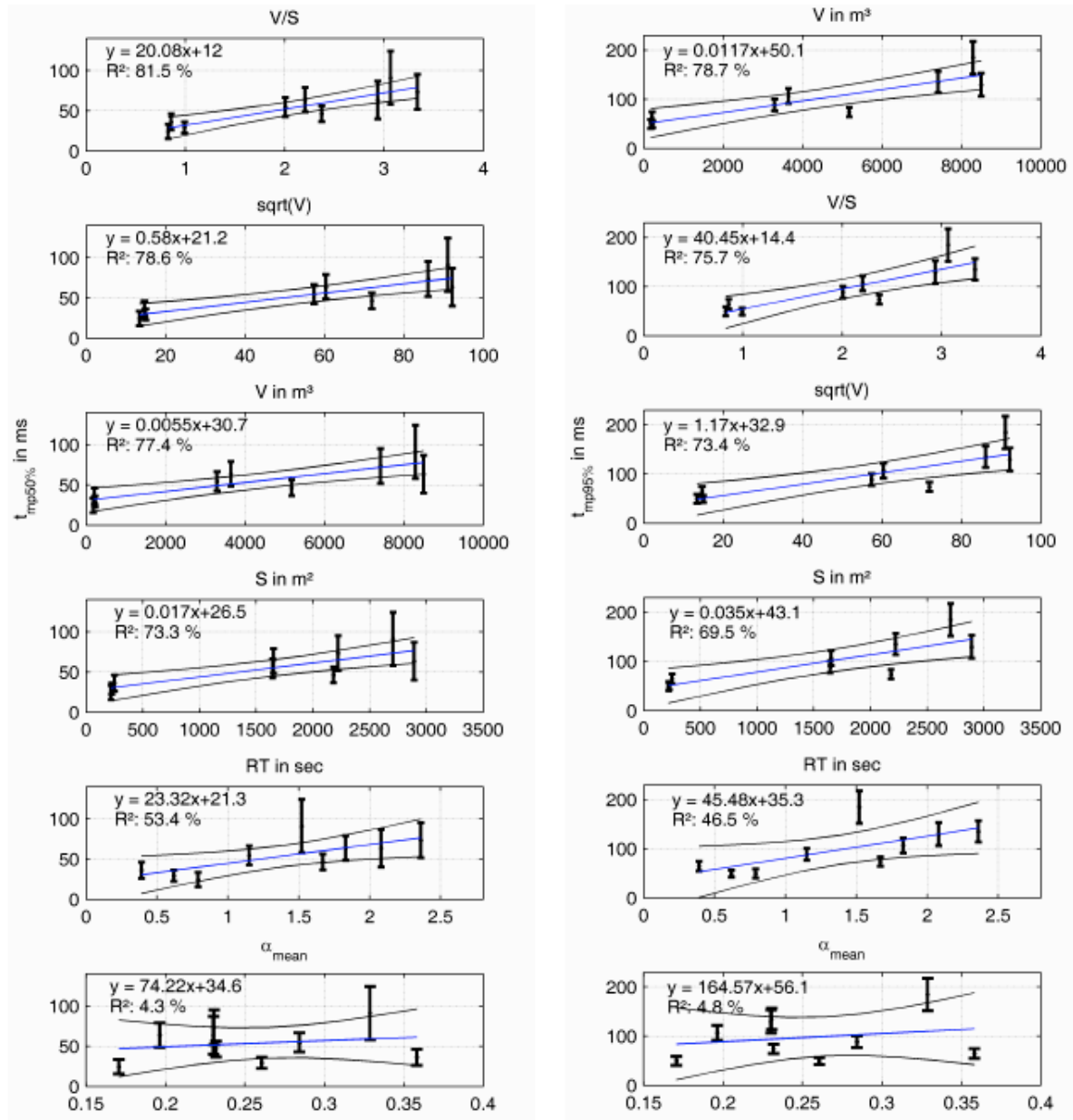


Figure 4.3.2.1. Linear regression analysis of the various model predictors and individual variables for $t_{mp50\%}$ (left cascade) and $t_{mp95\%}$ (right cascade) with 95% CI.

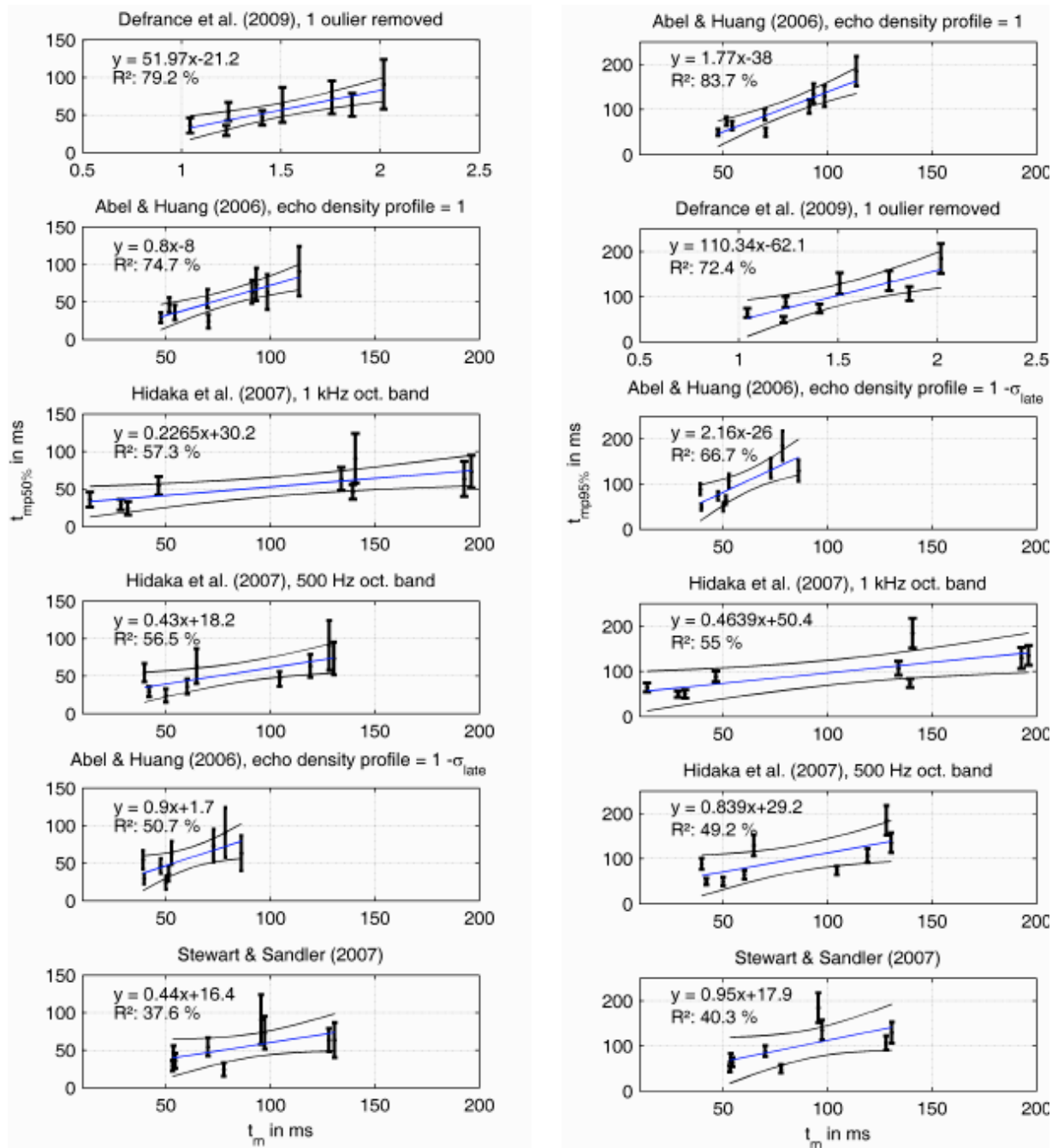


Figure 4.3.2.2. Linear regression analysis of the various empirically derived predictors for $t_{mp50\%}$ (left cascade) and $t_{mp95\%}$ (right cascade), with 95% CI.

	Model based predictors		Empirical predictors	
#	$T_{mp50\%}$	$T_{mp95\%}$	$T_{mp50\%}$	$T_{mp95\%}$
1	V/S R^2 81.5%	V R^2 78.7%	Abel I R^2 74.7%	Abel I R^2 83.7%
2	$\sqrt{V}$ R^2 78.6%	V/S R^2 75.7%	Hidaka 1k R^2 57.3%	Abel II R^2 66.7%
3	V R^2 77.4%	$\sqrt{V}$ R^2 73.4%	Hidaka 500 R^2 56.5%	Hidaka 1k R^2 55%
4	S R^2 73.3%	S R^2 69.5%	Abel II R^2 50.7%	Hidaka 500 R^2 49.2%
5	RT R^2 53.4%	RT R^2 46.5%	Stewart R^2 37.6%	Stewart R^2 40.3%
6	α_{mean} R^2 4.3%	α_{mean} R^2 4.8%	Defrance R^2 79.2%*	Defrance R^2 72.4%*

Table 4.3.2.2: Summary of correlation results for $t_{mp50\%}$ and $t_{mp95\%}$ against predictors. Red indicates an R^2 below 40%, orange indicates R^2 between 40% and 60%, yellow indicates an R^2 between 60% and 80%, and an R^2 over 80% is shown in green.

5 METHOD

It was intended that the fractal dimension (FD) of room impulse responses (RIRs) be investigated for correlation with known values for the perceptual mixing time (t_{mp}) for a set of rooms. Lindau's study was chosen for investigation because not only did the study provide values for t_{mp} for a variety of acoustic environments of known dimensions and average absorption coefficients (α_{ave}) from which the rooms could be modelled, but it also presented results for the gamut of mixing time predictors against which the effectiveness of this method could be measured.

To investigate how the FD of RIRs correlates with t_{mp} , models for the first eight rooms from the Lindau study were created based on Allen & Berkley's [1979] image-source method (ISM) in MATLAB⁴ [see Appendix 1 for code]. The ninth room from the study was excluded as it deviated from the typical shoebox shape (see Figure 4.3.1.1) and as such, is beyond the modelling capability of the ISM.

How the FD varied over the lifetime of a RIR was of great interest for this study. To obtain this FD 'profile' of a RIR, a function called *HMW.m* (see Appendix 1.2) was created in MATLAB that generated a moving window of a pre-defined sample length, which moves incrementally along the length of the RIR. As the moving window advances along the RIR, it implements the Higuchi algorithm (*hfd.m* – See appendix 1.3 for MATLAB code) for calculating the FD over the sample length, resulting in a profile showing how the FD of the RIR changes over its length. *HMW.m* was applied to the eight modelled RIRs from the Lindau study to produce eight FD profiles.

Following the production of the eight FD profiles, a mathematical analysis was carried out on each profile relating the behaviour of the profiles to the respective mean perceptual mixing times ($t_{mp50\%}$) from Lindau's study. The results from this analysis were then compared against Lindau's results for predictors of $t_{mp50\%}$ to determine the benefit, if any,

⁴ <http://www.mathworks.com/products/matlab/>

from the examination of the fractal dimension of RIRs as a predictor for the perceptual mixing time.

5.1 Image Source Method

The ISM implemented in MATLAB employed a re-coding of Fortran⁵ code (see Appendix 1.4.1 & 1.4.2 for MATLAB code) available in the appendix of Allen & Berkley's original ISM paper [Allen & Berkley, 1979]. ISM allows for the input of source and receiver position (in Cartesian coordinates) as well as for individual reflection coefficients, β , for each wall, the sample frequency and length of the resultant RIR. Figure 5.1a shows a typical setup of the MATLAB GUI (see Appendix 1.5.1 & 1.5.2 for MATLAB code) to generate an RIR.

When generating the eight RIRs modelled on Lindau's study a sample frequency of 44.1kHz was chosen. Only an average absorption coefficient (α_{ave}) for the whole of each room was available from the Lindau study, so this value was converted to β and inputted as each wall's coefficient. The receiver for each room was placed at length, $x =$ (half the length of the original room), width, $y = 1m$, and height, $z = 1.9m$, where x , y , and z represent length, width and height respectively. The source for each room was placed at $x =$ (half the length of the original room), $y = 2m$, and $z = 1.9m$.

Figure 5.1b shows the RIR created using ISM, modelled on Room 1 from Lindau's study.

5.2 Fractal Dimension Profile of Room Impulse Responses

The FD profiles for each RIR were created using the *HMW.m* function with a window length of 50 samples (~1ms). This value was chosen as it provided a highly detailed profile (see Figure 5.2) while being large enough for the Higuchi algorithm to work effectively (see Figure 3.1.2b). Before the FD profile is plotted it is smoothed with a moving window of 200 samples to remove noise. In Figure 5.2 the FD profile for the ISM modelled RIR for room 1 from Lindau's study can be seen.

⁵ <http://en.wikipedia.org/wiki/Fortran>

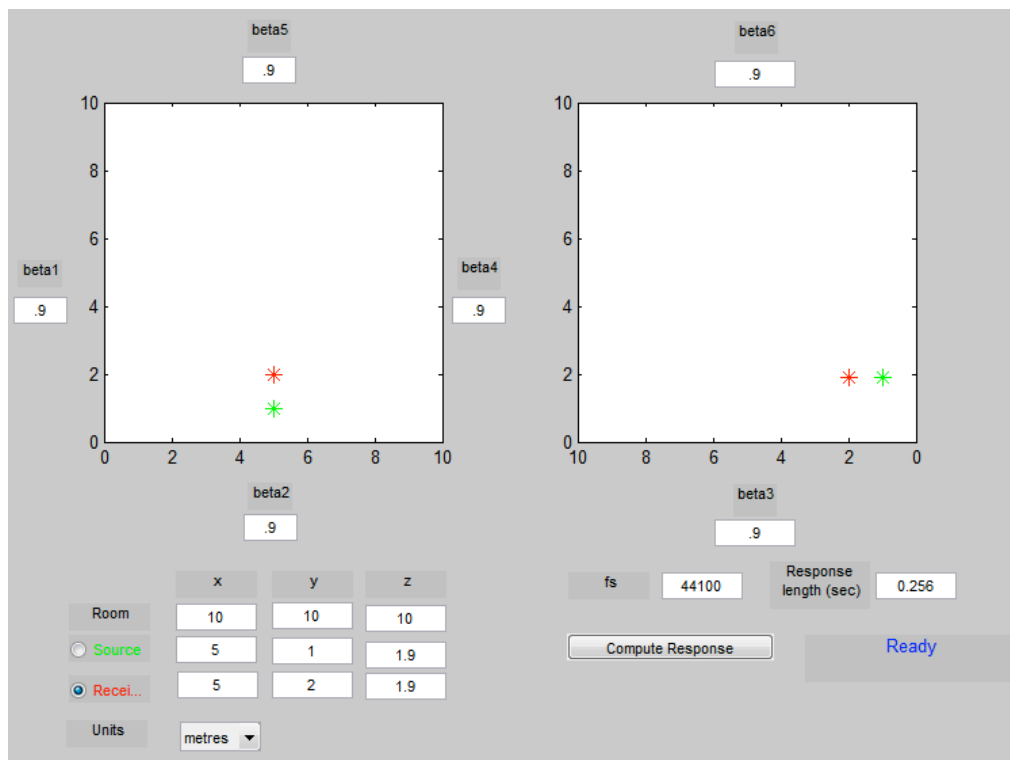


Figure 5.1a: GUI for ISM in MATLAB, showing typical setup to produce an RIR.

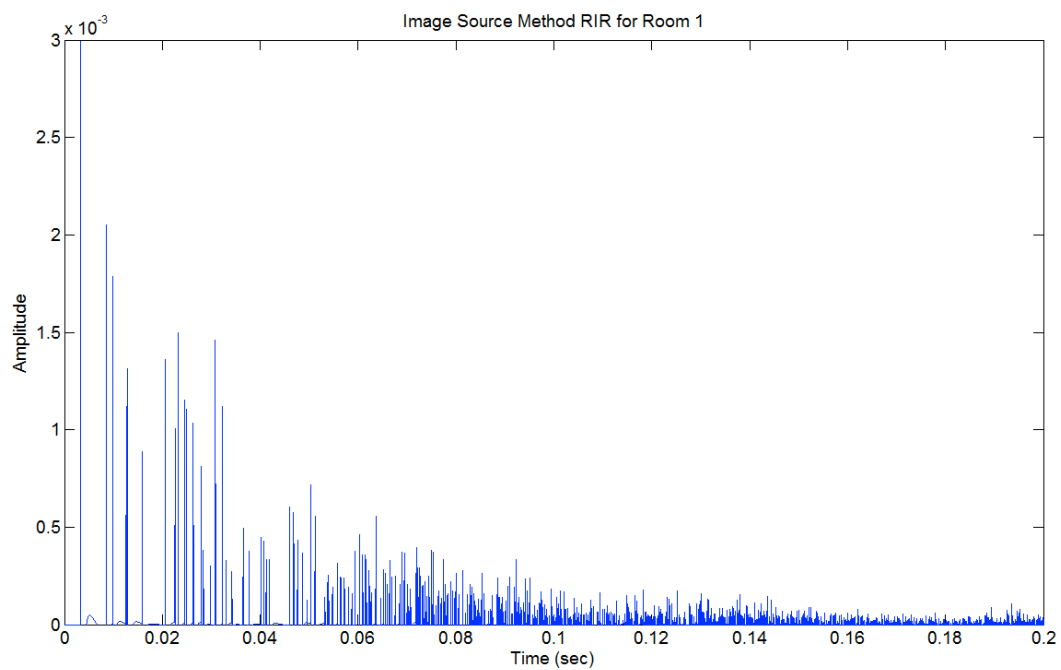


Figure 5.1b: ISM generated RIR for Room 1 from Lindau study.

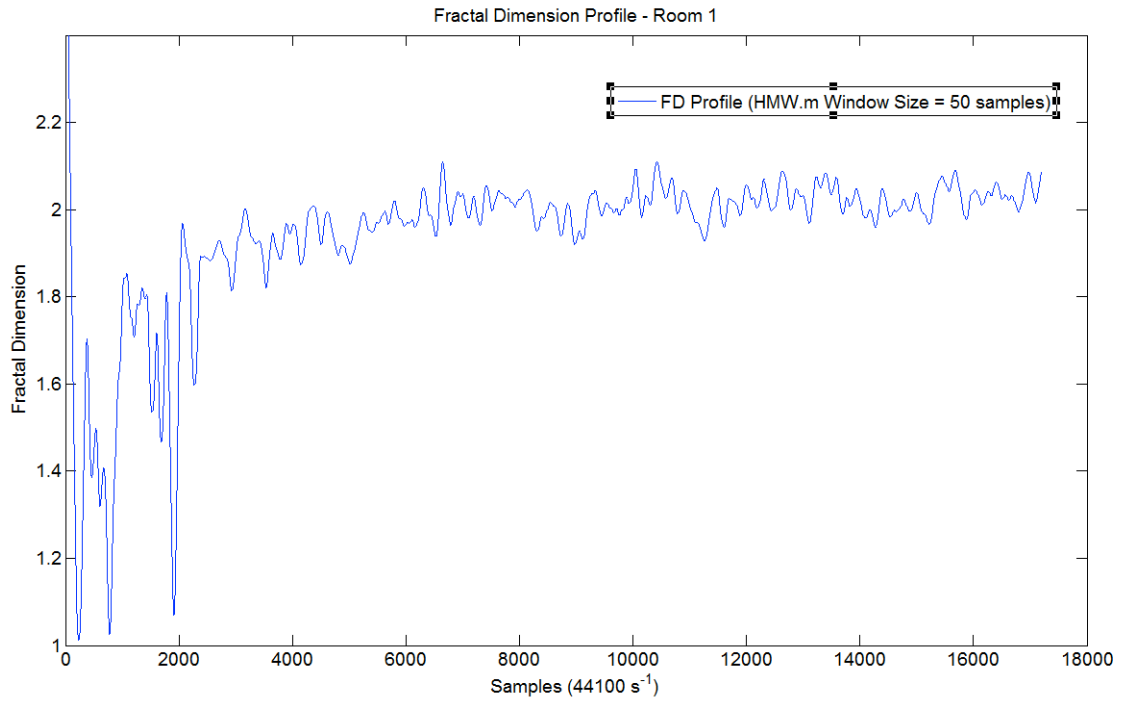


Figure 5.2: Fractal Dimension Profile from RIR of Room 1 created using *HMW.m.*, with a window size of 50 samples.

5.3 Analysis

The eight FD profiles were investigated to see if there was a correlation between the shape of the profile and $t_{mp50\%}$ from the Lindau study. It was presumed from diffusion theory (Section 2.2) that the RIR would be most diffuse towards its end. On this presumption the mean and standard deviation for the last 10% of each FD profile were calculated. Once these values had been established, four criteria relating to the FD profiles were created for investigating the relationship between the shape of the curves and $t_{mp50\%}$. Criterion I specified the sample number at which the magnitude of the FD profile first reaches the mean value calculated from the last 10% of its length. Criteria II, III & IV specified the sample number at which the magnitude of the FD profile first reaches the mean value calculated from the last 10% of its length minus 1, 2 and 3 times the standard deviation of the last 10% of its length, respectively. Figure 5.3 shows $t_{mp50\%}$ and the thresholds for criteria I – IV on the FD profile for Room 1.

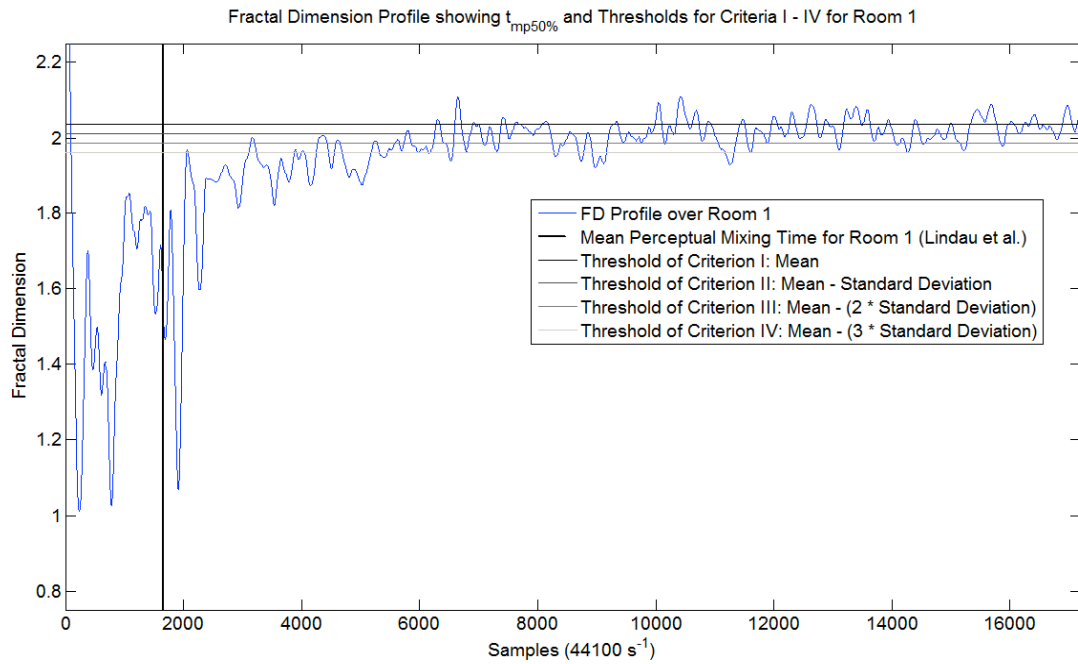


Figure 5.3: FD profile for Room 1 including $t_{mp50\%}$ and thresholds for Criteria I – IV.

Once the various criteria thresholds had been established for each room, the corresponding sample numbers were derived from each room using a MATLAB function, *FDthres.m* (Appendix 1.1). With the sample numbers obtained for each criterion, these values were plotted against $t_{mp50\%}$ from the Lindau study and a linear regression analysis conducted to test for correlation.

6 RESULTS

6.1 Criteria I – IV

The values for the mean and standard deviation for the last 10% of each room's fractal dimension (FD) profile and the FD thresholds for the four criteria from Section 5.3 are presented in Table 6.1a. From this table it can be seen that the largest mean FD value was 2.0450 for Room 2 and with the smallest value 1.9644 for Room 8. The standard deviation ranged from 0.0252 for Room 1 up to 0.0506 for Room 8. These results produced an FD threshold for Criterion I ranging from 2.0450 to 1.9644 (Room 2 and Room 8 respectively). For Criterion II the FD threshold ranges from 2.0185 to 1.9138 (Room 2 and Room 8 respectively). For Criterion III the FD threshold ranges from 1.9921 to 1.8631 (Room 2 and Room 8 respectively), and for Criterion IV the FD threshold ranges from 1.9656 to 1.8125 (Room 2 and Room 8 respectively).

From the values in Table 6.1a the sample numbers from the eight FD profiles for Criteria I – IV were obtained using *FDthres.m* (see Appendix 1.1). These sample numbers, and their corresponding time (*ms*) are presented in Table 6.1b. From the table it can be seen that the sample numbers (and their corresponding time) for Criterion I range from 4571 (103.70ms) in Room 2 to 16194 (367.21) in Room 8. For Criterion II the values range from 3197 (72.49 ms) in Room 2 to 12571 (285.06 ms) in Room 8. For Criterion III the values range from 2494 (56.55 ms) in Room 3 to 11708 (265.49 ms) in Room 7. For Criterion IV the sample values range from 1567 (35.53 ms) in Room 3, to 7661 (173.72 ms) in Room 5.

Room No.	Mean (FD)	Standard Deviation (FD)	Threshold (Criterion I) (FD)	Threshold (Criterion II) (FD)	Threshold (Criterion III) (FD)	Threshold (Criterion IV) (FD)
1	2.0364	0.0252	2.0364	2.0112	1.986	1.9608
2	2.0450	0.0265	2.0450	2.0185	1.9921	1.9656
3	2.0250	0.0321	2.0250	1.9927	1.9605	1.9284
4	1.9833	0.0394	1.9833	1.9439	1.9046	1.8652
5	1.9782	0.0404	1.9782	1.9377	1.8973	1.8569
6	1.9866	0.0432	1.9866	1.9435	1.9003	1.8571
7	1.9682	0.0394	1.9682	1.9288	1.8894	1.8500
8	1.9644	0.0506	1.9644	1.9138	1.8631	1.8125

Table 6.1a: Values of mean and standard deviation for the last 10% of the FD profiles of Rooms 1 – 8.

Room	Sample Number (Criterion I)	Time (ms) (Criterion I)	Sample Number (Criterion II)	Time (ms) (Criterion II)	Sample Number (Criterion III)	Time (ms) (Criterion III)	Sample Number (Criterion IV)	Time (ms) (Criterion IV)
1	6277	142.34	5772	130.88	3136	71.11	2053	46.55
2	4573	103.70	3197	72.49	3170	71.88	3049	69.14
3	6056	137.53	4335	98.30	2494	56.55	1567	35.53
4	13724	311.20	7027	160.36	7013	159.02	7004	158.82
5	7996	181.32	7971	180.75	7906	179.27	7661	173.72
6	12366	280.41	9486	215.10	8060	182.77	2052	46.53
7	14850	336.73	12308	279.09	11708	265.49	2192	49.71
8	16194	367.21	12571	285.06	8465	191.95	5209	118.12

Table 6.1b: Sample number (44100 samples s⁻¹) from the eight FD profiles for the four threshold criteria.

6.2 Correlation with Mean Perceptual Mixing Time

From the values in Table 6.1b, linear regression analysis was performed using MATLAB's curve fitting toolbox for the four criteria against $t_{mp50\%}$ from the Lindau study. Figure 6.2 shows plots of the sample values obtained from the four criteria against $t_{mp50\%}$. The linear regression equations and explained variance values (R^2) for the four criteria against $t_{mp50\%}$ are presented in Table 6.2.

From Figure 6.2 and Table 6.2 it can be seen that Criterion I produced a correlation with $t_{mp50\%}$, with an explained variance (R^2) of 72.29%. Criterion II produced a correlation with $t_{mp50\%}$, with an R^2 of 78.82%. Criterion III produced the best correlation with $t_{mp50\%}$, with an R^2 of 93.49% and a linear regression equation of the form,

$$t_{mp50\%} = 0.3197 \cdot III + 325, \quad (6.2)$$

where III indicates the sample number at the point the FD profile first crosses the FD threshold imposed by Criterion III. Criterion IV produced no correlation with $t_{mp50\%}$, with an R^2 of 0.14%.

It would be expected for a strong correlation, as observed for Criterion III, to have little or no constant in its linear regression equation because if the criterion value is zero so should $t_{mp50\%}$ be. This is observed in Equation 6.2, with the constant of 325 samples introducing an uncertainty of less than 1ms.

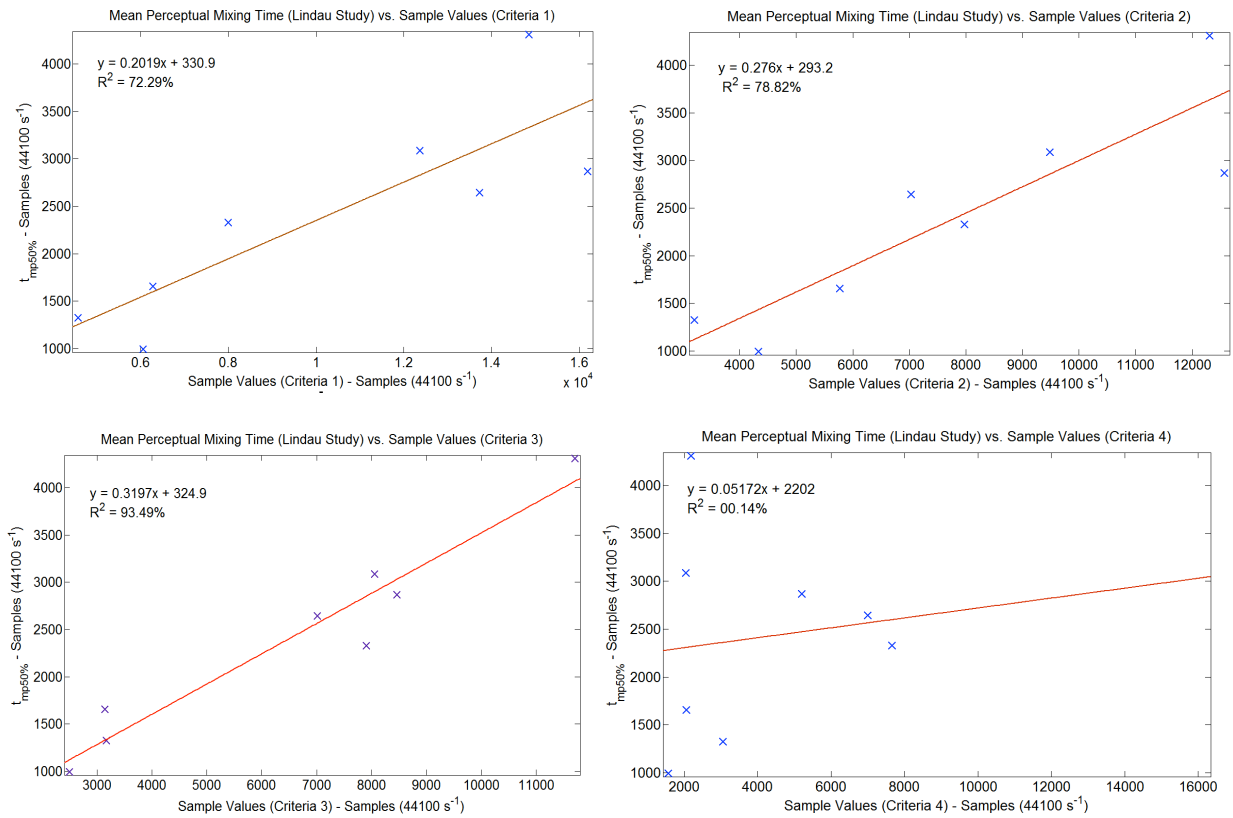


Figure 6.2: Plots of $t_{mp50\%}$ against sample values from Criteria I – IV with regression equation and R^2 values.

Criterion	Linear Regression Equation (Samples: 44100s ⁻¹)	Explained Variance (R^2)
I	$t_{mp50\%} = 0.2090 \cdot I + 331$	72.29%
II	$t_{mp50\%} = 0.2760 \cdot II + 293$	78.82%
III	$t_{mp50\%} = 0.3197 \cdot III + 325$	93.49%
IV	$t_{mp50\%} = 0.0517 \cdot IV + 2202$	00.14%

Table 6.2: Linear regression equations and explained variance for Criteria I – IV.

Rank	Model Predictors	Empirical Predictors
#	$t_{mp50\%}$	$t_{mp50\%}$
1	V/S $R^2 = 81.5\%$	Conaty III $R^2 = 93.49\%$
2	$\sqrt{V}$ $R^2 = 78.6\%$	Conaty II $R^2 = 78.82\%$
3	V $R^2 = 77.4\%$	Abel & Huang I $R^2 = 74.7\%$
4	S $R^2 = 73.3\%$	Conaty I $R^2 = 72.29\%$
5	RT $R^2 = 53.4\%$	Hidaka 1k $R^2 = 57.3\%$
6		Hidaka 500 $R^2 = 56.5\%$
7		Abel & Huang II $R^2 = 50.7\%$
8		Stewart & Sandler $R^2 = 37.6\%$

Table 6.3: Summary of correlation results for $t_{mp50\%}$ against predictors. Red indicates an R^2 below 40%, orange indicates an R^2 between 40% and 60%, yellow indicates an R^2 between 60% and 80%, and an R^2 over 80% is shown in green.

7 DISCUSSION & CONCLUSIONS

In this investigation an analysis of the fractal dimension (FD) profile of room impulse responses (RIRs) was investigated as a potential tool for predicting the perceptual mixing time (t_{mp}) of room impulse responses for use in hybrid artificial reverberation. RIRs were created from models of eight acoustic environments of shoebox shape, which were previously used in a study evaluating the performance of various model- and RIR- based predictors for t_{mp} [Lindau et al., 2010]. These RIRs were created using the image source method [Allen & Berkley, 1979]. Fractal dimension profiles of each of these RIRs were generated using the Higuchi algorithm [Higuchi, 1988] for estimating FD. The eight FD profiles were compared to mean perceptual mixing time ($t_{mp50\%}$) values from the evaluation study, which had been obtained from listening tests, under four criteria. Under the boundaries of the third criterion a correlation between the FD profile and $t_{mp50\%}$ was found, with an explained variance (R^2) of 93.4% and a linear regression equation of the form,

$$t_{mp50\%} = 0.3197 \cdot III + 325, \quad (6.2)$$

where III indicates the sample number at the point the FD profile first crosses the FD threshold imposed by Criterion III.

Methods for predicting the mixing time (t_{mix}) that look at RIRs have possibly failed in the past [Abel & Huang, 2006; Hidaka, 2007; Stewart & Sandler, 2007; Defrance et al, 2009] because by basing an analysis on the statistical behaviour of the reflections in the room, the behaviour (and limitations) of the human cognitive and auditory systems are not being fully taken into account. Model-based methods for predicting t_{mp} (see equations 4.3.2.1a-d) may have lacked accuracy, as they may not have considered all the factors contributing to the perception of mixing in a room. The results from this investigation suggest that the complexity of reflections in an acoustic environment, i.e.,

the amount of similarity over time, might contribute to the perception of mixing in an acoustic environment.

This method for determining t_{mp} offers a way to measure the complexity of an RIR, and, by the establishment of a criterion from experiment [Lindau et al., 2010] to equate human perception with the complexity of a RIR, provides a marked improvement over previous prediction methods [Lindau et al., 2010], with the added advantage that the FD profile of any room can easily and quickly be derived from a measured or modelled RIR.

7.1 Further Work

This approach to predicting the perceptual mixing time could be advanced with a comprehensive listening test. Such a test might ask the listener to discriminate between samples of audio that have either been treated with hybrid artificial reverberation using convolution with a real RIR or using convolution with a real RIR up to t_{mp} , followed by an algorithmically generated reverberation tail.

As this investigation used results from only one receiver position within each room, a further investigation that looked at the FD profile of RIRs from various receiver positions in the same room could be looked at. It may also be of value to investigate whether a large deviation in the FD of the impulse response between ears could be a factor affecting t_{mp} .

To better understand the factors that influence t_{mp} , in relation to the FD of an RIR, an understanding of the psychoacoustic response to the complexity is suggested.

8 BIBLIOGRAPHY

Abel, J.S., Huang, P., 2006, *A Simple, Robust Measure of Reverberation Echo Density*, Proceedings of the 121st Audio Engineering Society Convention, 6985.

Allen, J.B., Berkley, D.A., 1979, *Image method for efficiently simulating small-room acoustics*, Acoustical Society of America, 65 (4), 943-950.

Bracewell, R., 1965, *Convolution and Two-Dimensional Convolution*, The Fourier Transform and it's Applications, McGraw-Hill New York, 25-50 and 243-244.

Defrance, G., Daudet, L., Polack, J. D., 2009, *Using Matched Pursuit for Estimating Mixing Time Within Room Impulse Responses*, Acta Acustica united with Acustica, 95 (6), 1071 – 1081.

Douady, A., Hubbard, J.H., 1984, *Etude dynamique des polynômes complexes Partie I*, Publications Mathematiques d'Orsay, 84 (2).

Douady, A., Hubbard, J.H., 1985, *Etude dynamique des polynômes complexes Partie 2*, Publications Mathematiques d'Orsay, 85 (4).

Gardner, W.G., 1992. *The Virtual Acoustic Room*, Master of Science Thesis, Massachusetts Institute of Technology.

Gardner, W.G., 1995, *Efficient Convolution without Input-Output Delay*, *Journal of the Audio Engineering Society*, 43 (3), 127 – 136.

Gerzon, M.A., 1971, *Synthetic stereo reverberation Part I*, *Studio Sound*, 13, 632 - 635.

Gerzon, M.A., 1972, *Synthetic stereo reverberation Part II*, *Studio Sound*, 14, 24 - 28.

Giesbrecht, H., Mc Farland, W., Perry, T., 2009, *Algorithmic Reverberation: Combining Moorer's reverberator with simulated room impulse response reflection modelling*, ELEC 407 DSP Project, University of Victoria.

Griesinger, D., 1989, *Practical Processors and Programs for Digital Reverberation*, Proceedings of the 7th Audio Engineering Society Convention, 7027.

Hidaka, T., Yamada, Y., Nakagawa, T., 2007, *A new definition of boundary point between early reflections and late reverberation in room impulse responses*, *Journal of the Acoustical Society of America*, 22 (1), 326-332.

Higuchi, T., 1988, *Approach to an irregular time series on the basis of the fractal theory*, *Physica D*, 31, 277–283.

- Hirschman, I.I., Widder, D.V., 1955, *The Convolution Transform*, Princeton University Press, New Jersey.
- Hohlfield, R.G., 1999, *Self-similarity and the geometric requirements for frequency independence in antennae*, *Fractals*, 7 (1), 79 – 84.
- Jot, J.M., Chaigne, A., 1991, *Digital delay networks for designing artificial reverberators*, *Proceeding of the 90th Audio Engineering Society Convention*, 3030.
- Jot, J.M., 1992, *Etude et Réalisation d'un Spatialisateur de Sons par Modèles Physiques et Perceptifs*, PhD thesis, French Telecom, Paris, 92, 19.
- Kuttruff, H., 2000, *Room Acoustics*, Taylor & Francis, New York.
- Lindau, F., 2006, *An instrument for software-based measurement of binaural room impulse responses in multiple degrees of freedom*, *VDT International Convention*, 1-5.
- Lindau, A., Kosanke, L., Weinzierl, S., 2010, *Perceptual Evaluation of Physical Predictors of the Mixing Time in Binaural Room Impulse Responses*, *Proceedings of the 128th AES Convention*, 8089.
- Mandelbrot, B., 1967, *How Long Is the Coast of Britain? Statistical Self-Similarity and Fractional Dimension*, *Science*, 156, 3775, 636-638.
- Mandelbrot, B. B., 1982, *The Fractal Geometry of Nature*, W.H. Freeman and Company.
- Meesawat, K., Hammershoi, D., 2003, *The Time When The Reverberation Tail in a Binaural Room Impulse Response Begins*, *Proceedings of the 115th Audio Engineering Society Convention*, 5859.
- Moorer, J.A., 1979, *About this reverberation business*, *Computer Music Journal*, 3(2), 13 -18.
- Morse, P.M., Feshbach, H., 1953, *Methods of Theoretical Physics, Part I*, McGraw-Hill, New York, 464-465.
- Murphy, D., Stewart, R., 2007, *A Hybrid Artificial Reverberation Algorithm*, *Proceedings of the 122nd Audio Engineering Society Convention*, 7021.
- Polack, J.D., 1988, *La transmission de l'énergie sonore dans les salles*, Thèse de Doctorat d'Etat. Le Mans, Université du Maine.
- Polack, J.D., 1992, *Modifying Chambers to play Billiards; the Foundations of Reverberation Theory*, *Acustica*, 76, 257-272.
- Press, W.H.; Flannery, B.P.; Teukolsky, S.A., Vetterling, W.T., 1992, *Convolution and Deconvolution Using the FFT, Numerical Recipe*, in: *FORTRAN: The Art of Scientific Computing, 2nd ed.*, Cambridge University Press, 531-537.

Primavera, A. Palestini, L., Cecchi, S., Piazza, F., Moschetti, M., 2010, *A Hybrid Approach for Real-Time Room Acoustic Response Simulation*, Proceedings of the 128th Audio Engineering Society Convention, 8055.

Raghavendra, B.S., Dutt, D.N., 2009, *A note on Fractal Dimensions of Biomedical Waveforms*, Computers in Biology and Medicine, 39, 11.

Raghavendra, B.S. and Dutt, D.N, 2010, *Computing Fractal Dimension of Signals using Multiresolution Box Counting Method*, International Journal of Information and Mathematical Sciences, 6 (1), 50 - 52

Rubak, P., Johansen, L.G., 1999, *Artificial Reverberation based on a Pseudo-random Impulse Response II*, Proceedings of the 106th Audio Engineering Society Convention, 4900.

Sabine, W.C., 1993, *Collected Papers on Acoustics*, The Journal of the Acoustical Society of America, 95 (6), 367.

Schreiber, L., 1960, *Was empfinden wir als gleichförmiges uschen?*, Frequenz, 14 (12), 399.

Schroeder, M.R., 1962, *Natural Sounding Artificial Reverberation*, Journal of the Audio Engineering Society, 10 (3), 219-223.

Schroeder, M., 1989, *Self-Similarity and Fractals in Science and Art*, Journal of the Audio Engineering Society, 37 (10), 795 - 808.

Stan, G. B.; Embrechts, J.J; Archambeau, D., 2002, *Comparison of different impulse response measurement techniques*, Journal of the Audio Engineering Society, 50 (4), 249, 262.

Stewart, R., Sandler, M., 2007, *Statistical Measures of Early Reflections of Room Impulse Responses*, Proceedings of the 10th International Conference on Digital Audio Effects, 59.

Taylor, R., 2011, *There's more to implants than meets the eye*, Physics World, 24 (5), 22-27.

9 APPENDIX

9.1 MATLAB Code

9.1.1 FDthresh.m

```
%FDthresh.m - Thom Conaty
%Outputs the threshold values for the four Criterion from section 5.3
%Outputs sample number along profile at which Criterion n is met

function [i] = FDthresh(x,n)
    %x = FD profile data
    %n = Criterion number to determine threshold

    xtenpercent=round(length(x)/10) %Find 10% of profile to nearest sample
    xlasttenpercent=x(:, (length(x)-xtenpercent):length(x)); %Find last 10%
    of profile to

    xmeanlasttenpercent=mean(xlasttenpercent) %Find mean of last 10%
    xSDlasttenpercent=std(xlasttenpercent) %Find standard Deviation of last
    10%

    FDthres1=(xmeanlasttenpercent) %Criterion I
    FDthres2=(xmeanlasttenpercent-(1*xSDlasttenpercent)) %Criterion II
    FDthres3=(xmeanlasttenpercent-(2*xSDlasttenpercent)) %Criterion III
    FDthres4=(xmeanlasttenpercent-(3*xSDlasttenpercent)) %Criterion IV

    if n == 1

        for i=1000:1:length(x) %Start at sample 1000 to avoid FD error at
            start of profile

                i;
                x(i);
                if x(i)>=FDthres1
                    break
                end
            end
        end

    elseif n == 2

        for i=1000:1:length(x) %Start at sample 1000 to avoid FD error at
            start of profile

                i;
                x(i);
                if x(i)>=FDthres2
```

```

        break

    end
end

elseif n == 3

    for i=1000:1:length(x) %Start at sample 1000 to avoid FD error at
start of profile

        i;
        x(i);
        if x(i)>=FDthres3
            break

        end
    end

elseif n == 4

    for i=1000:1:length(x) %Start at sample 1000 to avoid FD error at
start of profile

        i;
        x(i);
        if x(i)>=FDthres4
            break

        end
    end

end

end

```

9.1.2 HMW.m

```

% HMW.m - Thom Conaty
% Creates moving window of size y samples that incrementally moves
through RIR data (of m channels) applies Higuchi Algorithm
% and applies Higuchi algorithm for calculating FD(hfd.m) over window.

function [FD] = HMW(x,y)
                                %x is Room Impulse Response (RIR) data to be
analysed
                                %y is Window Size

m=size(x,2); %Get number of channels
z=zeros(y,m); %add space at end of array

```

```

k=cat(1,x,z);% Add window of data to end of waveform for each channel

for j=1:m %proceed for each channel

    for i=1:1:length(k)%Move through waveform in each channel

        TraceSeg = k(i:(i+y),j); %Sample Section of waveform; length y

        if length(TraceSeg) + i + 1 > length(k) %Break if end of
waveform reached
            break

        end

        l(i,j)=hfd(TraceSeg); %Hyguci FD analysis Function
        i %Show movement through Waveform

    end

    sl=smooth(l,200) %smooth FD profile
    plot(sl) %Plot FD profile

end

```

9.1.3 hfd.m

```

% hfd - Salai Selvam V, 2011 - vsalaiselvam@yahoo.com
% Higuchi algorithm for calculating Fractal Dimension

function [varargout]=hfd(x,kmax)
% INPUT: x - time series for HFD is to be estimated; kmax - maximum
value of k
% OUTPUT: xhfd - HFD of x; lnk - ln(1/k); lnLk - ln(L(k)) log of average
curve length, L; Lmk - all values of curve length, L

if ~exist('kmax','var')||isempty(kmax),
    kmax=5;
end;

N=length(x);

Lmk=zeros(kmax,kmax);
for k=1:kmax,
    for m=1:k,
        Lmki=0;
        for i=1:fix((N-m)/k),
            Lmki=Lmki+abs(x(m+i*k)-x(m+(i-1)*k));
        end;
        Ng=(N-1)/(fix((N-m)/k)*k);
        Lmk(m,k)=(Lmki*Ng)/k;%Here is the problem in the code by Mr.
Tikkuhirvi & Mr. Aino
    end;
end;

```

```

Lk=zeros(1,kmax);
for k=1:kmax,
    Lk(1,k)=sum(Lmk(1:k,k))/k;
end;

lnLk=log(Lk);
lnk=log(1./[1:kmax]);

b=polyfit(lnk,lnLk,1);
xhfd=b(1);

varargout={xhfd,lnk,lnLk,Lk,Lmk};

% Reference:
% 1. T. Higuchi (1988), Approach to an irregular time series on the
basis of the fractal theory, Physica D, 277-283
% 2. C E Polychronaki et al (2010), Comparison of fractal dimension
% estimation algorithms for epileptic seizure onset detection, J. Neural
Eng. 7 (2010) 046007 (18pp)
% Copyright: V Salai Selvam, Tamil Nadu, India email:
% vsalaiselvam@yahoo.com

```

9.1.4 Image Source Method Matlab code

9.1.4.1 ithimage.m

```

%Image-Source Method MATLAB code - part 1 - Dermot Furlong -
dfurlong@tcd.ie
%See "Image method for efficiently simulating small-room acoustics"
%Allen and Berkley, J.Acoust.Soc.Am April '79
%This is a matlab version of their fortran code

function delp=lthimage(dr,dr0,r1,nr)

    r2l=zeros(1,3);delp=zeros(1,8);rp=zeros(3,8);
    %don't need other dimension statements
    %Loop over all sign permutations and compute r+/-r0
    i0=1;
    for l=-1:2:1
        for j=-1:2:1
            for k=-1:2:1
                % nearest image is l=j=k=-1
                rp(1,i0)=dr(1)+l*dr0(1);
                rp(2,i0)=dr(2)+j*dr0(2);
                rp(3,i0)=dr(3)+k*dr0(3);
                i0=i0+1;
            end %for k
        end %for j
    end %for l        %equivalent of label '10'

    % add in mean radius to 8 vectors to get total delay
    r2l(1)=2*r1(1)*nr(1);
    r2l(2)=2*r1(2)*nr(2);
    r2l(3)=2*r1(3)*nr(3);

```

```

for i=1:8
    delsq=0;
    for j=1:3
        r1=r2l(j)-rp(j,i);
        delsq=delsq+r1.^2;
    end %for j=1:3 %label is 25
    delp(i)=round(sqrt(delsq)); %want integer value for delp(i)
end %for i %label is '20'

return

```

9.1.4.2 sroom.m

```

%Image-Source Method MATLAB code - part 2 - Dermot Furlong -
dfurlong@tcd.ie
%See "Image method for efficiently simulating small-room acoustics"
%Allen and Berkley, J.Acoust.Soc.Am April '79
%This is a matlab version of their fortran code

function ht=sroom(r,r0,r1,beta,npts)

    ht=zeros(1,npts);
    %don't need dimension statement for r,r0,r1,
    delp=zeros(1,8);
    nr=zeros(1,3);
    %no such thing as equivalence in Matlab- whenever we assign to
    nx,ny,or nz we must also assign to nr
    %don't need to clear out ht

    dis=sum((r-r0).^2);
    dis=floor(sqrt(dis));

    %Check for mic and source at same location
    if (dis<.5)
        ht(1)=1;           %room resp is just a delta function
        return;           %exit sroom function
    end;

    %Find range of sum
    n1=floor(npts/(r1(1)*2)+1);
    n2=floor(npts/(r1(2)*2)+1);
    n3=floor(npts/(r1(3)*2)+1);

    for nx=-n1:n1
        for ny=-n2:n2
            for nz=-n3:n3
                nr=[nx ny nz];%no equivalence statement in Matlab
                %Get 8 image locations for mode #nr
                delp=lthimage(r,r0,r1,nr);
                i0=0;
                for l=0:1
                    for j=0:1
                        for k=0:1
                            i0=i0+1;
                            %make delay an integer

```

```

        id=delp(i0);%+.5;
        fdm1=id;
        id=id+1;
        if (id<=npts) %no goto in matlab
            %put in loss factor once foreach wall
reflection
            gid=(beta(1,1).^abs(nx-
1))*(beta(2,1).^abs(nx))*(beta(1,2).^abs(ny-j))*...
(beta(2,2).^abs(ny))*(beta(1,3).^abs(nz-k))*(beta(2,3).^abs(nz))./fdm1;
            %check for floating point underflow here
- don't have to do this in Matlab
            id;
            ht(id)=ht(id)+gid; %problem with out of
bounds here I think
        end
    end %for k
end %for j
end %for j %equalent of label '10'
end %for nz
end %for ny
end %for nz %equalent of label '20'

[B,A] = BUTTER(3,.01,'high');
ht=filter(B,A,ht); %filter room response
return

```

9.1.5 Image Source Method GUI

9.1.5.1 sroomgui.m

```

%Image-Source Method GUI MATLAB code - part 1 - Dermot Furlong -
dfurlong@tcd.ie

function fig = sroomgui()
% This is the machine-generated representation of a Handle Graphics
object
% and its children. Note that handle values may change when these
objects
% are re-created. This may cause problems with any callbacks written to
% depend on the value of the handle at the time the object was saved.
% This problem is solved by saving the output as a FIG-file.
%
% To reopen this object, just type the name of the M-file at the MATLAB
% prompt. The M-file and its associated MAT-file must be on your path.
%
% NOTE: certain newer features in MATLAB may not have been saved in this
% M-file due to limitations of this format, which has been superseded by
% FIG-files. Figures which have been annotated using the plot editor
tools
% are incompatible with the M-file/MAT-file format, and should be saved
as
% FIG-files.

%set(0,'RecursionLimit',4000);
close all;
load sroomgui

```

```

h0 = figure('Color',[0.8 0.8 0.8], ...
    'Colormap',mat0, ...
    'FileName','sroomgui.m', ...
    'PaperPosition',[18 180 576 432], ...
    'PaperUnits','points', ...
    'Position',[49 42 864 619], ...
    'Tag','Fig1', ...
    'ToolBar','none');
h1 = axes('Parent',h0, ...
    'Units','pixels', ...
    'Box','on', ...
    'ButtonDownFcn','sroomguicbs axis1mouse', ...
    'CameraUpVector',[0 1 0], ...
    'CameraUpVectorMode','manual', ...
    'Color',[1 1 1], ...
    'ColorOrder',mat1, ...
    'Position',[75 300 250 250], ...
    'Tag','Axes1', ...
    'XColor',[0 0 0], ...
    'XLim',[0 10], ...
    'XLimMode','manual', ...
    'YColor',[0 0 0], ...
    'YLim',[0 10], ...
    'YLimMode','manual', ...
    'ZColor',[0 0 0]);
h2 = line('Parent',h1, ...
    'ButtonDownFcn','sroomguicbs axis1mouse', ...
    'Color',[1 0 0], ...
    'LineWidth',1, ...
    'Marker','*', ...
    'MarkerSize',10, ...
    'Tag','axis1receiverpoint', ...
    'XData',[7 7], ...
    'YData',[7 7]);
h2 = line('Parent',h1, ...
    'Color',[0 1 0], ...
    'LineWidth',1, ...
    'Marker','*', ...
    'MarkerSize',10, ...
    'Tag','axis1sourcepoint', ...
    'XData',[5 5], ...
    'YData',[5 5]);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[4.979919678714859 -0.9638554216867465
17.32050807568877], ...
    'Tag','Axes1Text4', ...
    'VerticalAlignment','cap');
set(get(h2,'Parent'),'XLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[-1.004016064257028 4.939759036144579 17.32050807568877],
...
    'Rotation',90, ...
    'Tag','Axes1Text3', ...

```

```

        'VerticalAlignment','baseline');
set(get(h2,'Parent'),'YLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','right', ...
    'Position',mat2, ...
    'Tag','Axes1Text2', ...
    'Visible','off');
set(get(h2,'Parent'),'ZLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[4.979919678714859 10.28112449799197 17.32050807568877],
    ...
    'Tag','Axes1Text1', ...
    'VerticalAlignment','bottom');
set(get(h2,'Parent'),'Title',h2);
h1 = axes('Parent',h0, ...
    'Units','pixels', ...
    'Box','on', ...
    'ButtonDownFcn','sroomguicbs axis2mouse', ...
    'CameraUpVector',[0 1 0], ...
    'CameraUpVectorMode','manual', ...
    'Color',[1 1 1], ...
    'ColorOrder',mat3, ...
    'Position',[425 300 250 250], ...
    'Tag','Axes2', ...
    'XColor',[0 0 0], ...
    'XDir','reverse', ...
    'XLim',[0 10], ...
    'XLimMode','manual', ...
    'YColor',[0 0 0], ...
    'YLim',[0 10], ...
    'YLimMode','manual', ...
    'ZColor',[0 0 0]);
h2 = line('Parent',h1, ...
    'Color',[1 0 0], ...
    'LineWidth',1, ...
    'Marker','*', ...
    'MarkerSize',10, ...
    'Tag','axis2receiverpoint', ...
    'XData',[7 7], ...
    'YData',[7 7]);
h2 = line('Parent',h1, ...
    'Color',[0 1 0], ...
    'LineWidth',1, ...
    'Marker','*', ...
    'MarkerSize',10, ...
    'Tag','axis2sourcepoint', ...
    'XData',[5 5], ...
    'YData',[5 5]);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[5.020080321285143 -0.9638554216867465
17.32050807568877], ...
    'Tag','Axes2Text4', ...
    'VerticalAlignment','cap');

```

```

set(get(h2,'Parent'),'XLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[11.00401606425703 4.939759036144579 17.32050807568877],
    ...
    'Rotation',90, ...
    'Tag','Axes2Text3', ...
    'VerticalAlignment','baseline');
set(get(h2,'Parent'),'YLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','right', ...
    'Position',mat4, ...
    'Tag','Axes2Text2', ...
    'Visible','off');
set(get(h2,'Parent'),'ZLabel',h2);
h2 = text('Parent',h1, ...
    'Color',[0 0 0], ...
    'HandleVisibility','off', ...
    'HorizontalAlignment','center', ...
    'Position',[5.020080321285143 10.28112449799197 17.32050807568877],
    ...
    'Tag','Axes2Text1', ...
    'VerticalAlignment','bottom');
set(get(h2,'Parent'),'Title',h2);
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[4.875 289.875 30 15], ...
    'String','.9', ...
    'Style','edit', ...
    'Tag','b1');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[248.25 289.875 30 15], ...
    'String','.9', ...
    'Style','edit', ...
    'Tag','b4');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[132.1875 422.25 30 15], ...
    'String','.9', ...
    'Style','edit', ...
    'Tag','b5');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[132.75 169.5 30 15], ...
    'String','.9', ...
    'Style','edit', ...
    'Tag','b2');
h1 = uicontrol('Parent',h0, ...

```

```

        'Units','points', ...
        'BackgroundColor',[1 1 1], ...
        'ListboxTop',0, ...
        'Position',[393.75 420 45 15], ...
        'String','.7', ...
        'Style','edit', ...
        'Tag','b6');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[393.75 166.5 45 15], ...
    'String','.7', ...
    'Style','edit', ...
    'Tag','b3');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[96 82.5 45 15], ...
    'String','7', ...
    'Style','edit', ...
    'Tag','receiverx', ...
    'Value',1);
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[148.5 82.5 45 15], ...
    'String','7', ...
    'Style','edit', ...
    'Tag','receivery');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[95.25 102 45 15], ...
    'String','5', ...
    'Style','edit', ...
    'Tag','sourcex');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[148.5 101.25 45 15], ...
    'String','5', ...
    'Style','edit', ...
    'Tag','sourcey');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs roomdimensions', ...
    'ListboxTop',0, ...
    'Position',[148.5 120.75 45 15], ...
    'String','10', ...
    'Style','edit', ...
    'Tag','roomy');

```

```

h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs roomdimensions', ...
    'ListboxTop',0, ...
    'Position',[95.25 120 45 15], ...
    'String','10', ...
    'Style','edit', ...
    'Tag','roomx');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[200.25 81.75 45 15], ...
    'String','7', ...
    'Style','edit', ...
    'Tag','receiverz');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs sreditboxchange', ...
    'ListboxTop',0, ...
    'Position',[200.25 99 45 15], ...
    'String','5', ...
    'Style','edit', ...
    'Tag','sourcez');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'Callback','sroomguicbs roomdimensions', ...
    'ListboxTop',0, ...
    'Position',[200.25 119.25 45 15], ...
    'String','10', ...
    'Style','edit', ...
    'Tag','roomz');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'Callback','sroomguicbs toggle_r', ...
    'ListboxTop',0, ...
    'Position',[35.625 120 45 15], ...
    'String','Room', ...
    'Style','text', ...
    'Tag','StaticText1');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'Callback','sroomguicbs toggle_r', ...
    'ListboxTop',0, ...
    'Position',[148.5 138 45 15], ...
    'String','y', ...
    'Style','text', ...
    'Tag','StaticText5');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'Callback','sroomguicbs toggle_r', ...

```

```

        'ListboxTop',0, ...
        'Position',[200.25 138 45 15], ...
        'String','z', ...
        'Style','text', ...
        'Tag','StaticText6');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'Callback','sroomguicbs toggle_r', ...
    'ListboxTop',0, ...
    'Position',[95.25 138 45 15], ...
    'String','x', ...
    'Style','text', ...
    'Tag','StaticText8');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[7.5 309.75 24.75 15], ...
    'String','beta1', ...
    'Style','text', ...
    'Tag','beta1');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[251.25 308.25 24 18], ...
    'String','beta4', ...
    'Style','text', ...
    'Tag','beta4');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[135.1875 439.5 24 18], ...
    'String','beta5', ...
    'Style','text', ...
    'Tag','beta5');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[405 438.75 24 18], ...
    'String','beta6', ...
    'Style','text', ...
    'Tag','beta6');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[404.25 183 24 18], ...
    'String','beta3', ...
    'Style','text', ...
    'Tag','beta3');
h1 = uicontrol('Parent',h0, ...

```

```

        'Units','points', ...
        'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
        'ListboxTop',0, ...
        'Position',[135 186.75 24.75 15], ...
        'String','beta2', ...
        'Style','text', ...
        'Tag','beta2');
h1 = uicontrol('Parent',h0, ...
        'ListboxTop',0, ...
        'Style','text', ...
        'Tag','ZOOMFigureState', ...
        'UserData',mat5, ...
        'Visible','off');
h1 = uicontrol('Parent',h0, ...
        'Units','points', ...
        'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
        'Callback','sroomguicbs toggle_s', ...
        'ForegroundColor',[0 1 0], ...
        'ListboxTop',0, ...
        'Position',[35.625 102.75 45 15], ...
        'String','Source', ...
        'Style','radiobutton', ...
        'Tag','Sourcebutton');
h1 = uicontrol('Parent',h0, ...
        'Units','points', ...
        'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
        'Callback','sroomguicbs toggle_r', ...
        'ForegroundColor',[1 0 0], ...
        'ListboxTop',0, ...
        'Position',[35.625 80.25 45 15], ...
        'String','Receiver', ...
        'Style','radiobutton', ...
        'Tag','Recbutton', ...
        'Value',1);
h1 = uicontrol('Parent',h0, ...
        'Units','points', ...
        'BackgroundColor',[1 1 1], ...
        'ListboxTop',0, ...
        'Position',[364.5 137.25 45 15], ...
        'String','8000', ...
        'Style','edit', ...
        'Tag','fs');
h1 = uicontrol('Parent',h0, ...
        'Units','points', ...
        'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
        'ListboxTop',0, ...
        'Position',[312.75 137.25 45 15], ...
        'String','fs', ...
        'Style','text', ...
        'Tag','StaticText2');
h1 = uicontrol('Parent',h0, ...
        'Units','points', ...
        'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
        'FontSize',10, ...
        'ForegroundColor',[0 0 1], ...
        'ListboxTop',0, ...

```

```

        'Position',[444 91.5 117 26.25], ...
        'String','Ready', ...
        'Style','text', ...
        'Tag','dialogue');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[34.5 55.5 45 15], ...
    'String','Units', ...
    'Style','text', ...
    'Tag','StaticText4');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'Callback','sroomguicbs computeresponse', ...
    'ListboxTop',0, ...
    'Position',[312 103.5 114.75 15], ...
    'String','Compute Response', ...
    'Tag','Pushbutton1');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[1 1 1], ...
    'ListboxTop',0, ...
    'Position',[483 137.25 45 15], ...
    'String','0.256', ...
    'Style','edit', ...
    'Tag','responselength');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'BackgroundColor',[0.752941176470588 0.752941176470588
0.752941176470588], ...
    'ListboxTop',0, ...
    'Position',[424.5 132 55.5 26.25], ...
    'String','Response length (sec)', ...
    'Style','text', ...
    'Tag','StaticText7');
h1 = uicontrol('Parent',h0, ...
    'Units','points', ...
    'ListboxTop',0, ...
    'Position',[97.5 54.75 45 15], ...
    'String',mat6, ...
    'Style','popupmenu', ...
    'Tag','units', ...
    'Value',1);
if nargout > 0, fig = h0; end

```

9.1.5.2 sroomguicbs.m

```

%Image-Source Method GUI MATLAB code - part 2 - Dermot Furlong -
dfurlong@tcd.ie

function sroomguicbs(action)

calx=200;caly=425;ca2x=550;ca2y=425; %centres of the axes
maxaxispixels=250;%250x250 on screen
switch action

```

```

case 'roomdimensions'

    disp('roomdimensions');
    roomxp=findobj(gcf,'Tag','roomx');
    roomyp=findobj(gcf,'Tag','roomy');
    roomzp=findobj(gcf,'Tag','roomz');

    %(isempty(get(roomzp,'String')) | (isempty(get(roomyp,'String'))))

    if ( (isempty(get(roomxp,'String')) |
(isempty(get(roomyp,'String')) | (isempty(get(roomzp,'String')))) )
        disp('room dimensions incomplete');

    else
        roomx=eval(get(roomxp,'String'));
        roomy=eval(get(roomyp,'String'));
        roomz=eval(get(roomzp,'String'));
        %change the 'position' attrib of axis 1 and two
        ppunit=round(maxaxispixels/max([roomx roomy roomz]));
        %pixels per unit length

        axis1p=findobj(gcf,'Tag','Axes1');
        posv1(1)=calx-(ppunit*(roomx/2));
        posv1(2)=caly-(ppunit*(roomy/2));
        posv1(3)=roomx*ppunit;
        posv1(4)=roomy*ppunit;
        set(axis1p,'Position',posv1);
        set(axis1p,'XLim',[0 roomx]);
        set(axis1p,'YLim',[0 roomy]);

        axis2p=findobj(gcf,'Tag','Axes2');
        posv2(1)=ca2x-(ppunit*(roomy/2));
        posv2(2)=ca2y-(ppunit*(roomz/2));
        posv2(3)=roomy*ppunit;
        posv2(4)=roomz*ppunit;
        set(axis2p,'Position',posv2);
        set(axis2p,'XLim',[0 roomy]);
        set(axis2p,'YLim',[0 roomz]);

        betalp=findobj(gcf,'Tag','beta1');
        posvb1(1)=calx-(ppunit*(roomx/2))-75;
        posvb1(2)=caly;
        posvb1(3)=25;
        posvb1(4)=15;
        replotsandr
        %set(betalp,'Position',posvb1);
    end

case 'toggle_s'
    disp('toggle_s');
    rb=findobj(gcf,'Tag','Recbutton');
    rv=(get(rb,'Value'));

    if (rv==1)
        set(rb,'Value',0);
    else
        set(rb,'Value',1);
    end

```

```

end

case 'toggle_r'
    disp('toggle_r');
    sb=findobj(gcf,'Tag','Sourcebutton');
    sv=(get(sb,'Value'));

    if (sv==1)
        set(sb,'Value',0);
    else
        set(sb,'Value',1);
    end

case 'axis1mouse'

    set(gcf,'WindowButtonMotionFcn','sroomguicbs axis1move');
    set(gcf,'WindowButtonUpFcn','sroomguicbs axis1stop');
    sroomguicbs('axis1move')

case 'axis2mouse'

    set(gcf,'WindowButtonMotionFcn','sroomguicbs axis2move');
    set(gcf,'WindowButtonUpFcn','sroomguicbs axis2stop');
    sroomguicbs('axis2move')

case 'axis1move'
    sb=findobj(gcf,'Tag','Sourcebutton');
    sv=(get(sb,'Value'));

    if sv==1           %source button on
        sourcexp=findobj(gcf,'Tag','sourcex');
        sourceyp=findobj(gcf,'Tag','sourcey');
        currPt=get(gca,'CurrentPoint');
        roomxp=findobj(gcf,'Tag','roomx');
        roomyp=findobj(gcf,'Tag','roomy');
        roomx=eval(get(roomxp,'String'));
        roomy=eval(get(roomyp,'String'));
        currPt=currPt.*(currPt>=0); %make sure >=0
        set(sourcexp,'String',min(currPt(1,1),roomx));
        set(sourceyp,'String',min(currPt(1,2),roomy));

    else               %receiver button on
        receiverxp=findobj(gcf,'Tag','receiverx');
        receiveryp=findobj(gcf,'Tag','receivery');
        currPt=get(gca,'CurrentPoint');
        roomxp=findobj(gcf,'Tag','roomx');
        roomyp=findobj(gcf,'Tag','roomy');
        roomx=eval(get(roomxp,'String'));
        roomy=eval(get(roomyp,'String'));
        currPt=currPt.*(currPt>=0); %make sure >=0
        set(receiverxp,'String',min(currPt(1,1),roomx));
        set(receiveryp,'String',min(currPt(1,2),roomy));
    end
end

```

```

case 'axis2move'
    sb=findobj(gcf,'Tag','Sourcebutton');
    sv=(get(sb,'Value'));

    if sv==1           %source button on
        sourcezp=findobj(gcf,'Tag','sourcez');
        currPt=get(gca,'CurrentPoint');
        roomzp=findobj(gcf,'Tag','roomz');
        roomz=eval(get(roomzp,'String'));
        currPt=currPt.*(currPt>=0); %make sure >=0
        set(sourcezp,'String',min(currPt(1,2),roomz));
    else               %receiver button on
        receiverzp=findobj(gcf,'Tag','receiverz');
        currPt=get(gca,'CurrentPoint');
        roomzp=findobj(gcf,'Tag','roomz');
        roomz=eval(get(roomzp,'String'));
        currPt=currPt.*(currPt>=0); %make sure >=0
        set(receiverzp,'String',min(currPt(1,2),roomz));
    end

case 'axis1stop'
    replotsandr
    set(gcf,'WindowButtonMotionFcn','');
    set(gcf,'WindowButtonUpFcn','');

case 'axis2stop'
    replotsandr
    set(gcf,'WindowButtonMotionFcn','');
    set(gcf,'WindowButtonUpFcn','');

case 'sreditboxchange'
    disp('sreditboxchange');
    replotsandr

case 'computeresponse'
    disp('computeresponse');
    flag=0;
    [flag, roomx, roomy, roomz, sx, sy, sz, rx, ry, rz, b1, b2, b3, b4,
b5, b6, fs, respl,units]=checkallparams;
    if flag==0
        disp('problem with paramaters');
    else
        disp('paramaters okay');
        %testmfile
        flag
        roomx
        roomy
        roomz
        sx
        sy
        sz
        rx
        ry
        rz
        b1
        b2
        b3

```

```

b4
b5
b6
fs
respl
units

%
% r=[50 50 10];
% r0=[50 50 50];
% r1=[100 100 100];
% beta=[.9 .9 .9;.9 .9 .9];

%
% npts=200;
% ht=sroom(r,r0,r1,beta,npts);
r=zeros(1,3);r0=zeros(1,3);r1=zeros(1,3);beta=zeros(1,6);

if units==1
    factor=1;
elseif units==2
    factor=3.28;
end

r1(1)=roomx*fs*factor/1000;
r1(2)=roomy*fs*factor/1000;
r1(3)=roomz*fs*factor/1000;
r0(1)=sx*fs*factor/1000;
r0(2)=sy*fs*factor/1000;
r0(3)=sz*fs*factor/1000;
r(1)=rx*fs*factor/1000;
r(2)=ry*fs*factor/1000;
r(3)=rz*fs*factor/1000;
beta=[b1 b2 b3;b4 b5 b6];
npts=respl*fs;

r;
r0;
r1;
beta;
npts;

dialoguep=findobj(gcf,'Tag','dialogue');
set(dialoguep,'String','Computing response ...');

ht=sroom(r,r0,r1,beta,npts);
figure(11);
plot((0:npts-1)/fs,ht);
ylabel('Amplitude');
xlabel('Time,seconds');
title('Room response');
%save ht.mat ht;

dialoguep=findobj(gcf,'Tag','dialogue');
set(dialoguep,'String','Ready');

[filename, pathname] = uinputfile('userparams\*.mat', 'Save
results to file ?')
if (filename~=0)

```

```

        fullname=[pathname filename ];
        save(fullname);
    end

    %function ht=sroom(r,r0,r1,beta,npts)

end;

end;

function replotsandr;
    sourcexp=findobj(gcbf,'Tag','sourcecx');
    sourceyp=findobj(gcbf,'Tag','sourcecy');
    sourcezp=findobj(gcbf,'Tag','sourcecz');
    receiverxp=findobj(gcbf,'Tag','receiverx');
    receiveryp=findobj(gcbf,'Tag','receivery');
    receiverzp=findobj(gcbf,'Tag','receiverz');

    sx=eval(get(sourcexp,'String'));
    sy=eval(get(sourceyp,'String'));
    sz=eval(get(sourcezp,'String'));

    rx=eval(get(receiverxp,'String'));
    ry=eval(get(receiveryp,'String'));
    rz=eval(get(receiverzp,'String'));

    sp1=findobj(gcbf,'Tag','axis1sourcepoint');
    set(sp1,'XData',[sx sx]);
    set(sp1,'YData',[sy sy]);
    sp2=findobj(gcbf,'Tag','axis2sourcepoint');
    set(sp2,'XData',[sy sy]);
    set(sp2,'YData',[sz sz]);

    rp1=findobj(gcbf,'Tag','axis1receiverpoint');
    set(rp1,'XData',[rx rx]);
    set(rp1,'YData',[ry ry]);
    rp2=findobj(gcbf,'Tag','axis2receiverpoint');
    set(rp2,'XData',[ry ry]);
    set(rp2,'YData',[rz rz]);

function [flag, roomx, roomy, roomz, sx, sy, sz, rx, ry, rz, b1, b2, b3,
b4, b5, b6, fs, respl, units]=checkallparams()

    flag=1;

sx=0;sy=0;sz=0;rx=0;ry=0;rz=0;b1=0;b2=0;b3=0;b4=0;b5=0;b6=0;fs=0;r1=0;
    dialoguep=findobj(gcbf,'Tag','dialogue');
    roomxp=findobj(gcbf,'Tag','roomx');
    roomyp=findobj(gcbf,'Tag','roomy');
    roomzp=findobj(gcbf,'Tag','roomz');

    %(isempty(get(roomzp,'String')) | (isempty(get(roomyp,'String'))))

```

```

    if ( (isempty(get(roomxp, 'String')) |
(isempty(get(roomyp, 'String')) | (isempty(get(roomzp, 'String')) )
        set(dialoguep, 'String', 'Imcomplete room dimensions');
        flag=0;
        return;
    end

    roomx=eval(get(roomxp, 'String'));
    roomy=eval(get(roomyp, 'String'));
    roomz=eval(get(roomzp, 'String'));
    if (roomx<=0) | (roomy<=0) | (roomz<=0)
        set(dialoguep, 'String', 'Negative room dimensions');
        flag=0;
        return;
    end

    sourcexp=findobj(gcbox, 'Tag', 'sourcexp');
    sourceyp=findobj(gcbox, 'Tag', 'sourceyp');
    sourcezp=findobj(gcbox, 'Tag', 'sourcezp');
    receiverxp=findobj(gcbox, 'Tag', 'receiverxp');
    receiveryp=findobj(gcbox, 'Tag', 'receiveryp');
    receiverzp=findobj(gcbox, 'Tag', 'receiverzp');

    sx=eval(get(sourcexp, 'String'));
    sy=eval(get(sourceyp, 'String'));
    sz=eval(get(sourcezp, 'String'));
    rx=eval(get(receiverxp, 'String'));
    ry=eval(get(receiveryp, 'String'));
    rz=eval(get(receiverzp, 'String'));
    if (sx<0) | (sx>roomx) | (sy<0) | (sy>roomy) | (sz<0) | (sz>roomz)
        set(dialoguep, 'String', 'Source out of bounds');
        flag=0;
        return;
    end

    if (rx<0) | (rx>roomx) | (ry<0) | (ry>roomy) | (rz<0) | (rz>roomz)
        set(dialoguep, 'String', 'Receiver out of bounds');
        flag=0;
        return;
    end

    beta1p=findobj(gcbox, 'Tag', 'b1');
    beta2p=findobj(gcbox, 'Tag', 'b2');
    beta3p=findobj(gcbox, 'Tag', 'b3');
    beta4p=findobj(gcbox, 'Tag', 'b4');
    beta5p=findobj(gcbox, 'Tag', 'b5');
    beta6p=findobj(gcbox, 'Tag', 'b6');

    b1=eval(get(beta1p, 'String'));
    b2=eval(get(beta2p, 'String'));
    b3=eval(get(beta3p, 'String'));
    b4=eval(get(beta4p, 'String'));
    b5=eval(get(beta5p, 'String'));
    b6=eval(get(beta6p, 'String'));

    if
(b1<0) | (b1>1) | (b2<0) | (b2>1) | (b3<0) | (b3>1) | (b4<0) | (b4>1) | (b5<0) | (b5>1) | (b
6<0) | (b6>1)
        set(dialoguep, 'String', 'Beta value out of bounds');

```

```

        flag=0;
        return;
    end

    fsp=findobj(gcf,'Tag','fs');
    if isempty(get(fsp,'String'))
        set(dialoguep,'String','fs incomplete');
        flag=0;
        return;
    end
    fs=eval(get(fsp,'String'));
    if (fs<0)
        set(dialoguep,'String','fs negative');
        flag=0;
        return;
    end

    resplp=findobj(gcf,'Tag','responselength');
    if isempty(get(resplp,'String'))
        set(dialoguep,'String','Response length incomplete');
        flag=0;
        return;
    end

    respl=eval(get(resplp,'String'));
    if (respl<0)
        set(dialoguep,'String','Response length negative');
        flag=0;
        return;
    end

    unitsp=findobj(gcf,'Tag','units');
    if isempty(get(unitsp,'String'))
        set(dialoguep,'String','Unit choice incomplete');
        flag=0;
        return;
    end
    units=(get(unitsp,'Value'));

    if (units~=1)&(units~=2)
        set(dialoguep,'String','Unit choice incomplete');
        flag=0;
        return;
    end

    flag=1;      %if we get this far then then all paramaters have been
checked
    set(dialoguep,'String','Paramaters okay');

```

9.2 Convolution

9.2.1 Direct Time Domain Convolution

Convolution in the time domain can be modelled as filtering process with FIR architecture, and is described by

$$y(n) = x(n) * h(n) = \sum_{m=0}^{N-1} x(m)h(n-m), \quad (9.2.1a)$$

where $x(n)$ is the input sample, $h(n)$ is the RIR and $y(n)$ is the output reverberated signal.

The filter can be represented in a block diagram as shown in Figure 9.2.1. With an input signal of $N(x)$ samples and RIR of $N(h)$ samples the output signal will be $N(x) + N(h) - 1$ samples long. This convolution requires multiplication and addition of the input signal with the RIR so with an input of length 1 second and RIR of 2 seconds with a sample rate of 44.1 kHz, this operation would require ~3.89 billion operations. It is clear from Figure 9.2.1 that this type of brute force operation is computationally very demanding.

9.2.2 Direct Frequency Domain Convolution

As previously mentioned, an input signal can be translated from the time domain into the frequency as convolution in the time domain is equivalent to complex multiplication in the frequency domain. This is represented as

$$y(n) = x(n) * h(n) = X(z) \cdot H(z). \quad (9.2.2a)$$

This translation is accomplished by performing the mathematical operation of a Fast Fourier Transform (FFT), which is a highly efficient form of the Discrete Fourier Transform (DFT) on each input. The transform can be carried out very quickly and once in the frequency domain, the process of convolution simply requires that an Inverse DFT be taken of the product of the FFTs of the two inputs, such that,

$$y(n) = IDFT(FFT(x) \cdot FFT(h)) \quad (9.2.2b)$$

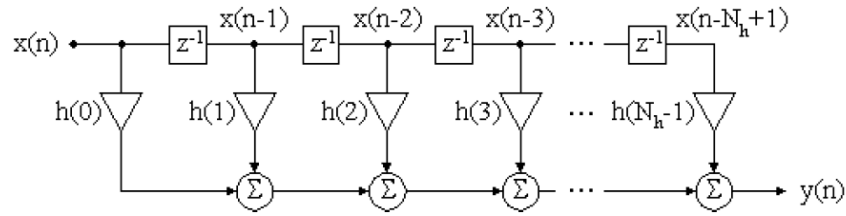


Figure 9.2.1: Block diagram representation of FIR filter for time domain convolution.

This simplification requires two $N(x) + N(h) - 1$ FFTs, $N(x) + N(h) - 1$ complex multiplications and one $N(x) + N(h) - 1$ IDFTs. Even with padding the input and impulse responses with zeros to the nearest power of 2 (a requirement for FFT), the total required computation will be less than 1% of that needed for time domain direct convolution. As previously mentioned, the whole length of the inputs must be present for direct frequency domain convolution so it cannot be used for real-time processing.

9.2.3 Block Convolution

Block convolution takes advantage of the real-time applicability of time domain convolution and the efficiency of frequency domain convolution. Three methods will be discussed in the following sub-sections: Overlap and Add, Overlap and Save, and Zero-Delay Convolution.

9.2.3.1 Overlap and Add

For this method the input signal is broken into smaller sections (or blocks) and convolved with an RIR in the frequency domain. The blocks are divided so they are non-overlapping and sequential. This allows for many shorter convolutions to take place over time. Convolution proceeds with the periodicity of the input blocks being exploited so that they can be processed through a special form of convolution known as *circular convolution*.

Circular convolution can be used when a function g_T is periodic, with period T . Then, for a function f , such that $f * g_T$ exists, whose convolution is also periodic and identical, the circular convolution of these functions is defined as

$$(f * g_T)(t) \equiv \int_{t_0}^{t_0+T} \left[\sum_{k=-\infty}^{\infty} f(\tau + kT) \right] g_T(t - \tau) d\tau \quad (9.2.3.1a)$$

where t_0 is an arbitrary time. This exploitation of the periodic nature of g_T saves on computational demand.

Once the separate convolutions have been executed, the resulting outputs are summed with overlapping by the input section length, N , minus one sample.

Figure 9.2.3.1a shows the example of an input signal, $x(n)$, of 9 samples and a RIR, $h(n)$, of 3 samples. There will be three output blocks, $y_m(n)$, from the three individual convolutions, as in Figure 9.2.3.1b. The summation of these blocks with overlap $N = 3 - 1 = 2$ results in an output of length of input signal length + RIR length - 1 = 9 + 3 - 1 = 11, as in Figure 9.2.3.1c.

After selecting the length of the input block, the input and RIR must be padded to maintain the output length for convolution in the frequency domain.

Although this form of block convolution exploits the highly efficient use of frequency domain convolution it still requires that the whole first block is loaded before convolution can begin, thus resulting in a small delay which may be tolerable for some applications.

9.2.3.2 Overlap and Save

The Overlap and Save method of convolution is similar to the Overlap and Add method except the input blocks are selected to overlap rather than the output. When the input blocks are being processed, the first $(N_k/2) - 1$ samples are discarded and each block concatenated to form the total output signal, $y(n)$.

Consider the following example (see Figure 9.2.3.2a); in this example the first two samples are discarded due to artefacts from the circular convolution (Figure 9.2.3.2b). The individual output blocks are then summed to produce the final output (Figure 9.2.3.2c).

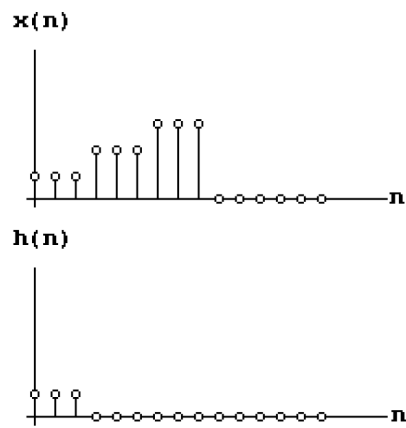


Figure 9.2.3.1a: Input signal and RIR for Overlap and Add example.

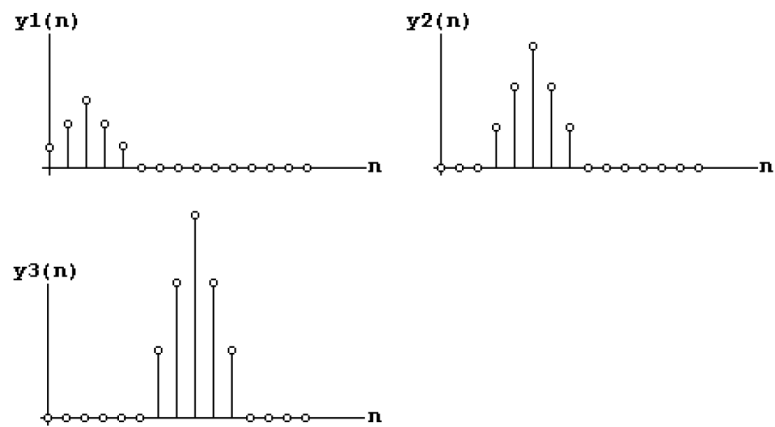


Figure 9.2.3.1b: Three output blocks from Overlap and Add example.

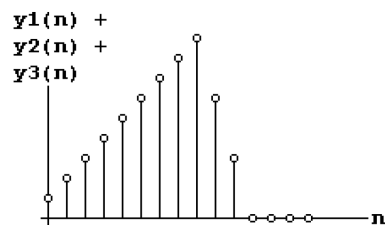


Figure 9.2.3.1c: Output from Overlap and Add example.

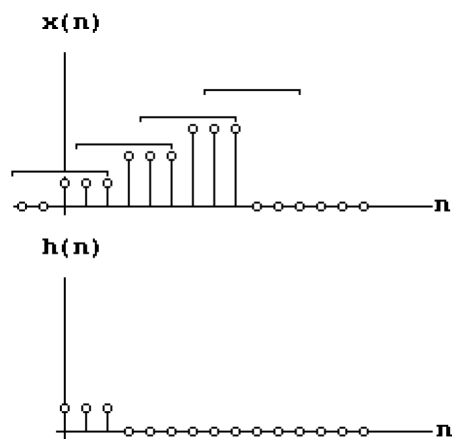


Figure 9.2.3.2a: Overlap and Save example showing input signal (divided into blocks, denoted by horizontal brackets) and RIR.

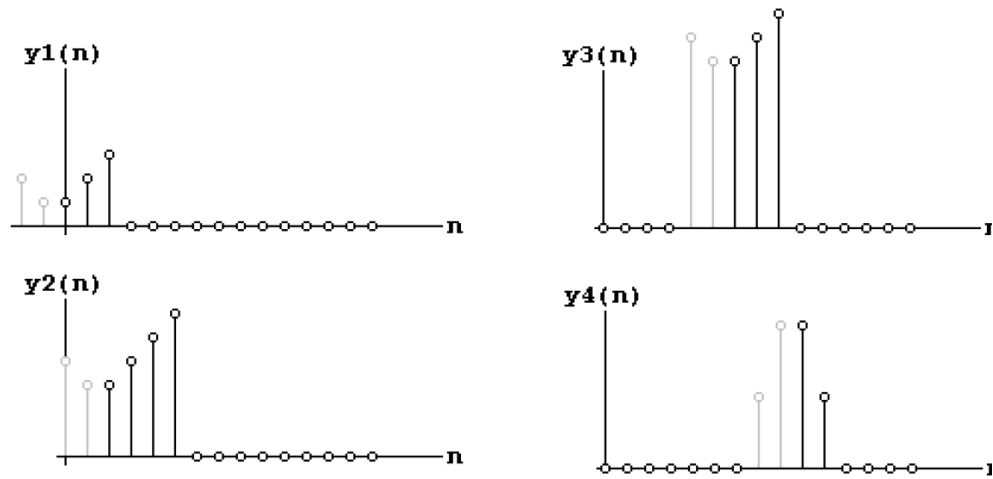


Figure 9.2.3.2b Output blocks from Overlap and Save example. Samples to be discarded before summation shown in gray.

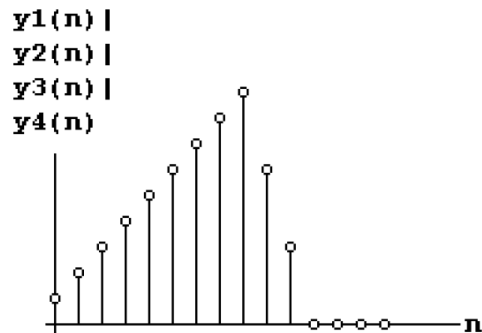


Figure 9.2.3.2c: Output from Overlap and Save example.

This method still introduces a delay but does not suffer from the accumulation step for the overlap sections incurred in the Overlap and Add method.

9.2.3.3 Zero-Delay Method

Gardiner has proposed a hybrid method of convolution that uses elements of FIR and block convolution that produces minimal delay in the output signal (Figure 9.2.3.3a) [Gardner 1995].

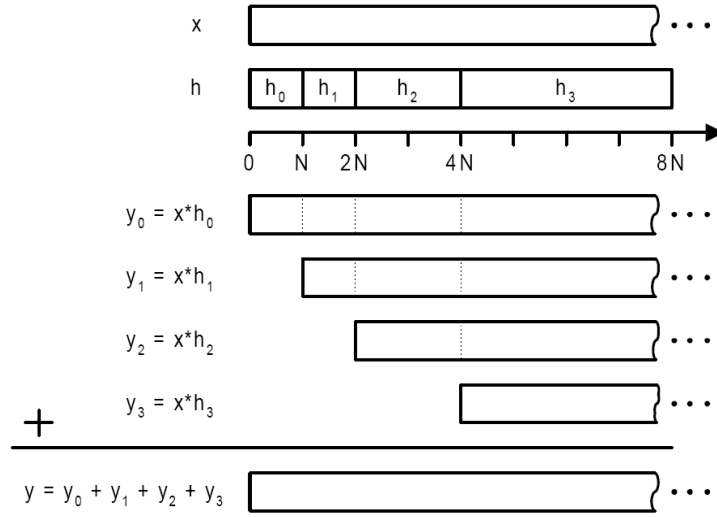


Figure 9.2.3.3a: Gardiner's zero-delay convolution algorithm.

This initial suggestion for a method of convolution by Gardiner works by decomposing the impulse response into successively larger sections (of the form $2^x N$) and for direct FIR convolution with the input signal. The first output, y_0 , is calculated by direct FIR convolution with the input signal. During this calculation there is enough time to collect N samples for a block convolution of the first N samples to produce y_1 . In the time for this calculation there is enough time to collect $2N$ samples for the next block convolution. This process continues for the whole input length. Due to the inability to increase the block size without violating the zero-delay constraint, Gardner rejected this method in preference to another algorithm that allows uniform processing.

This uniform processing is achieved by starting block convolution, with blocks of size N that start at least $2N$ samples into the filter response (Figure 9.2.3.3b).

Despite an amount of process scheduling, Gardner's revised algorithm results in zero-delay convolution.

9.3 Simulation Techniques

9.3.1 Image-Source Model – Mathematics

An ISM derived RIR, $h(t)$ is equated as

$$h(t) = \sum_{u=0}^1 \sum_{l=-\infty}^{\infty} A(u, \lambda) \cdot \delta(t - \tau(u, \lambda)) \quad (9.3.1a)$$

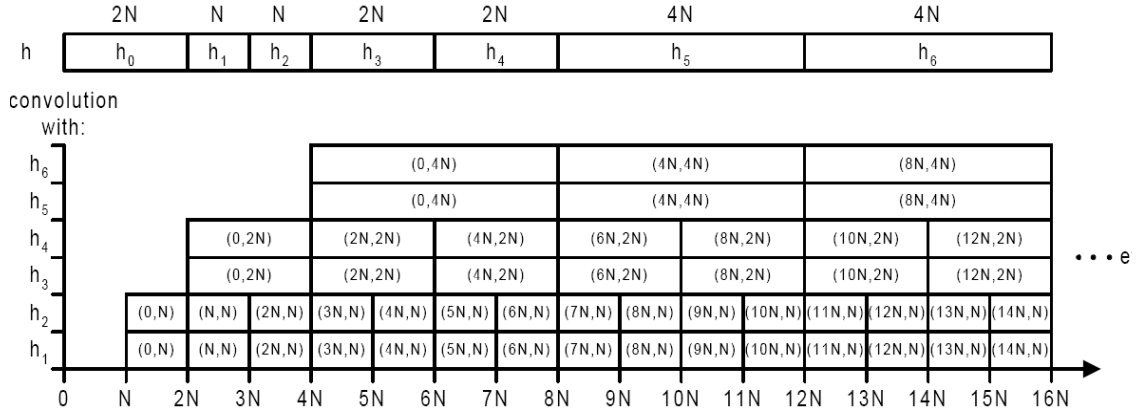


Figure 9.2.3.3b: Gardner's second proposed algorithm with a more practical decomposition of the impulse response with the corresponding convolution schedule.

where t denotes time, $\delta(\cdot)$ is the Dirac impulse function, the contribution of each image source to the receiver is a replica of the original source signal delayed by a lag time τ , and attenuated by an amplitude factor A . The triplets $\cup = (u, v, w)$ and $\lambda = (l, m, n)$ are parameters controlling the indexing of the image sources in all dimensions. For conciseness, the sum over $\cup$ (and respectively λ) in Equation 9.3.1a is used to represent the triple sum over each of the triplet's internal indices. The attenuation factor $A(\cdot)$ and time delay $\tau(\cdot)$ in Equation 9.3.1a are defined as,

$$A(\cup, \lambda) = \frac{\beta_{x,1}^{|\lambda-\cup|} \beta_{x,2}^{|\lambda|} \beta_{y,1}^{|m-v|} \beta_{y,2}^{|m|} \beta_{z,1}^{|n-w|} \beta_{xz,2}^{|n|}}{4\pi \cdot d(\cup, \lambda)}$$

$$\tau(\cup, \lambda) = d(\cup, \lambda) / c$$

where c is the speed of sound (ms^{-1}) and $\beta_{x,i}$, $\beta_{y,i}$ and $\beta_{z,i}$, are the reflection coefficients for each of the room's walls, where β is defined as

$$\beta = \left| \sqrt{1 - \alpha} \right|$$

where α is the absorption coefficient. $d(\cdot)$ represents the distance from the image to the receiver and is equated as

$$d(\cup, \lambda) = \left\| \text{diag}(2u-1, 2v-1, 2w-1) \cdot p_s + p_r - \text{diag}(2l, 2m, 2n) \cdot r \right\|$$

with $\| \cdot \|$ the Euclidian norm and $diag(\cdot)$ denoting a diagonal matrix with its arguments as diagonal elements, where p_s and p_r are the source and receiver positions in a Cartesian coordinate system (x, y, z) denoted by

$$p_s = [x_s, y_s, z_s]^T$$

$$p_r = [x_r, y_r, z_r]^T$$

and r is the vector of room dimensions, with length L_x width L_y and height L_z and is denoted as

$$r = [L_x, L_y, L_z]^T$$

where $[\cdot]^T$ denotes the matrix transpose operator.

9.4 Artificial Digital Reverberation – Mathematics

9.4.1 Analysis Techniques of Digital Filters

There are a number of methods for analysing the behaviour of filters in both the time and frequency domain. In the following sub-sections the different techniques are explained in detail and then applied to the two filters.

9.4.1.1 Difference Equation

The difference equation is a formula for computing an output sample at time n , based on past and present input samples in the time domain. The construction of the difference equation is best explained through the example of the feedback comb filter

$$y(n) = x(n) + gy(n - M)$$

where $x(n)$ denotes the n^{th} sample of the input signal, $y(n)$ is the output at time n , and g denotes the feedback coefficient.

9.4.1.2 System (Block) Diagram

A filter block diagram is a visual representation of the elements required to implement the filter. See Figure 9.4.1.2 for an example of a typical block diagram of a comb filter.

The symbol Z^m in the square box of Figure 9.4.1.2 denotes the input sample being held for a delay of m samples with the arrows denoting the flow of information. The $\oplus$ symbol indicates addition of the sample value with the input $x(n)$ and the $\times$ symbol indicates the multiplication of the sample value by g .

9.4.1.3 Impulse Response

The impulse response is a measurement of a filter's response to the Kronecker delta function, $\delta(n)$, and is defined as

$$\delta(n) = \begin{cases} 1, n = 0 \\ 0, n \neq 0 \end{cases}$$

and can be written

$$\delta = [1, 0, 0, \dots]$$

In a real-world system, the impulse $\delta(n)$ could be a gunshot or balloon pop.

The impulse response of a filter is defined as

$$h(n) = L_n \{ \delta(\cdot) \}$$

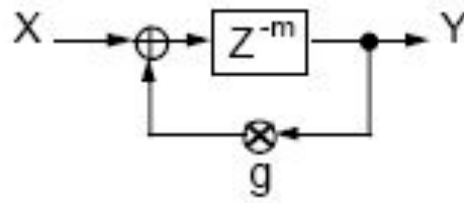


Figure 9.4.1.2: Block diagram of a comb filter.

where the impulse response $h(n)$ is the response of the filter L at time n to a unit impulse occurring at time 0 . A requirement of the filter is that it decays towards zero, otherwise the filter will become unstable.

9.4.1.4 Transfer Function

A transfer function provides an algebraic representation of a linear, time-invariant filter in the frequency domain. It is represented as a ratio of $Y(z)$ to $X(z)$, where $Y(z)$ denotes the z transform of the filter output signal $y(n)$, and $X(z)$ denotes the z transform of the filter input signal $x(n)$.

With $h(n)$ denoting the impulse response of the filter, the transfer function, $H(z)$, is equal to the z transform of $h(n)$ given by,

$$H(z) = \frac{Y(z)}{X(z)}.$$

9.4.1.5 Pole-Zero Plot

A pole-zero plot is a graphical representation of the transfer function, $H(z)$, in the z -plane. This analysis technique helps to reveal information regarding the stability and phase extremes of the system. As $H(z)$ is a ratio of polynomials of z , and since polynomials can be represented as their roots plus a scale factor, $H(z)$ can be characterised by its numerator roots (known as the *zeros* of the filter), its denominator roots (known as the *poles* of the filter), and a constant gain factor.

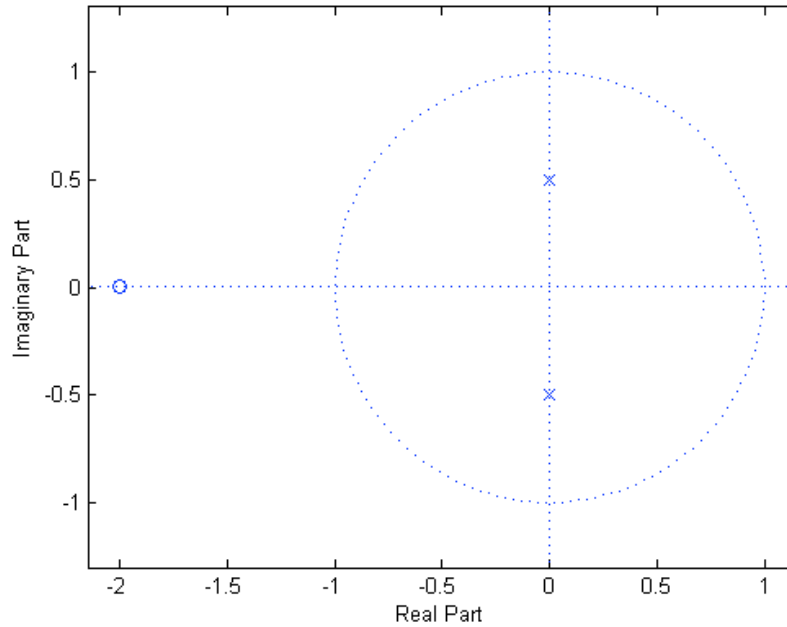


Figure 9.4.1.5: Zero-pole plot of transfer function in z -plane

Taking the example of $H(z) = \frac{z + 2}{z^2 + \frac{1}{4}}$, the sole *zero* of z is located at -2 with the *poles*

found at $\pm \frac{i}{2}$. These values are represented in a zero-pole plot in Figure 9.4.1.5 above.

On the zero-pole plot of Figure 9.4.1.5, the *zero* point is represented by the 'o' symbol and the poles by the 'x' markings. The magnitude of $H(z)$ will be larger when it is close to the poles and smaller when it is closer to the *zero(s)*. This provides a qualitative understanding of how the system behaves at various frequencies.

9.5 The Comb and All-Pass Filters in Detail

9.5.1 The Comb Filter

The comb filter (IIR type) comprises a simple delay and feedback line. This produces an exponentially decaying time domain impulse response and is analogous to continued reflections between two facing walls. The block diagram of a comb filter is shown in Figure 9.5.1a.

The difference equation for this filter is given by

$$y(n) = x(n) + gy(n - M)$$

where $x(n)$ denotes the n^{th} sample of the input signal, $y(n)$ is the output at time n , M is the length of the delay, and g denotes the feedback coefficient.

The filter has the transfer function:

$$H(z) = \frac{1}{1 - gz^{-M}}$$

The impulse response is described by

$$\begin{aligned} h(n) &= \delta(n) + g\delta(n - M) + g^2\delta(n - 2M) + g^3\delta(n - 3M) + \dots \\ &= \sum_{k=0}^{\infty} g^k \delta(n - kD) \end{aligned}$$

The term ‘comb filter’ is derived from the topology of the frequency response resulting from its implementation. This is seen in Figure 9.5.1b.

The zero-pole plot for the comb filter shows equally spaced poles around the unit circle with a radius specified by attenuation constant, g (see Figure 9.5.1c).

It is important that g is not > 1 as this will push the poles outside the unit circle and cause the filter to overload. The comb filter is susceptible to ‘flutter’ echoes that produce a ‘metallic’ timbre.

9.5.2 All-pass Filter

All-pass filters are used to maintain the time-dependence of a reverberation algorithm without influencing its frequency response. The construction of the All-pass is similar to that of a comb filter but achieves frequency independence with the use of a feedforward

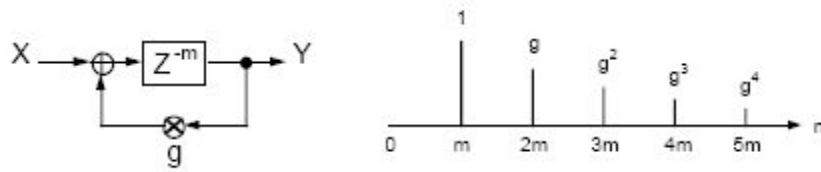


Figure 9.5.1a: Block diagram and impulse response of a comb filter.

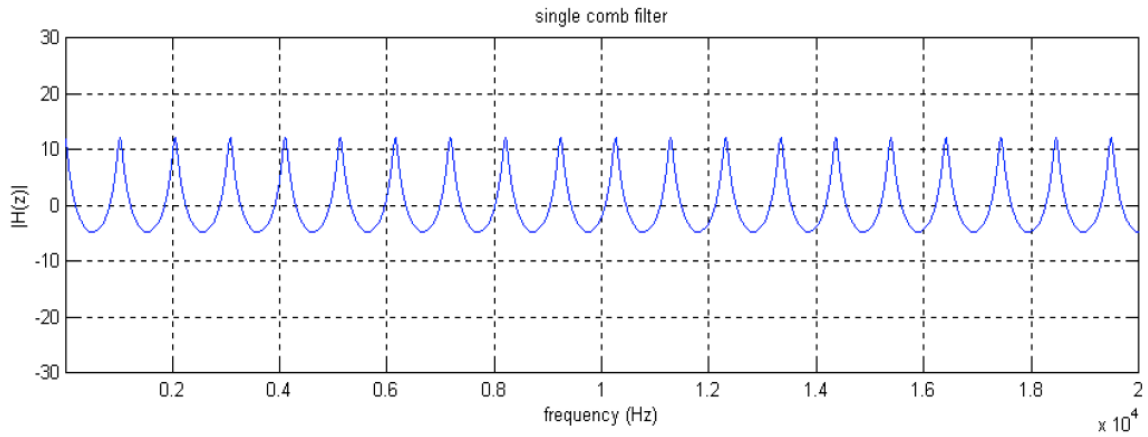


Figure 9.5.1b: Frequency response of comb filter.

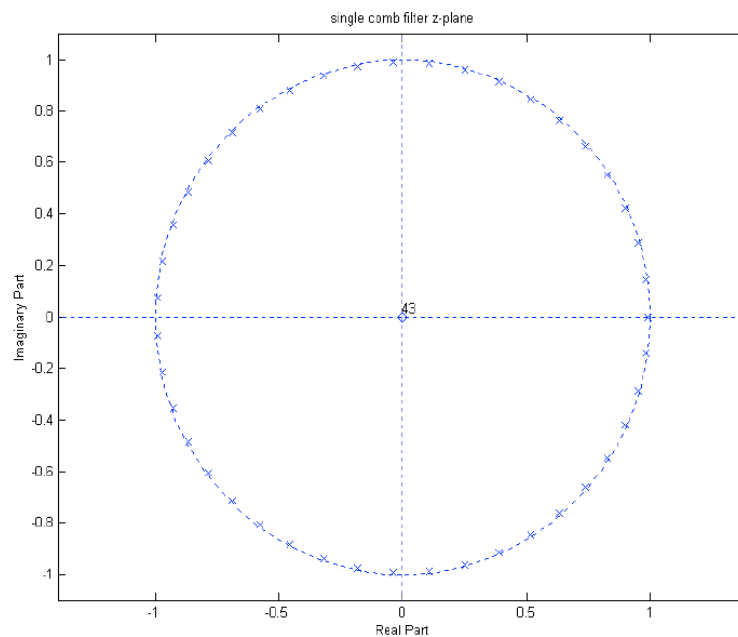


Figure 9.5.1c: Frequency response in z-plane.

path. This creates zeros at the same angle but with an inverse radius to the comb filter's poles, as seen in a zero-pole plot, cancelling the pole's influence on the magnitude of the frequency response of the filter.

An all-pass filter has the difference equation given by

$$y(n) = -gx(n) + x(n - M) + gy(n - M)$$

and transfer function given by

$$H(z) = \frac{-g + z^{-M}}{1 - gz^{-M}}$$

and impulse response

$$\begin{aligned} h(n) &= -g\delta(n) + (1 - g^2)\delta(n - M) + g(1 - g^2)\delta(n - 2M) + g^2(1 - g^2)\delta(n - 3M) + \dots \\ &= -g\delta(n) + (1 - g^2) \sum_{k=1}^{\infty} g^{k-1} \delta(n - kM). \end{aligned}$$

The block diagram of an all-pass filter can be seen in Figure 9.5.1a, and its impulse response in Figure 9.5.2b. Figure 9.5.2c shows the zero-pole plot for an all-pass filter.

The all-pass filter, although producing a flat frequency response overall, will have a non-flat frequency shift due to phase shift, short time intervals, and transient sounds. This gives the all-pass filter a unique and recognisable timbre.

9.5.3 Feedback Delay Networks

Feedback delay networks (FDN) were first suggested for use with ADR algorithms by Michael Gerzon after observing that individual feedback comb filters yielded poor quality sound whereas when several were cross-coupled the result was of superior quality. [Gerzon, 1971; Gerzon, 1972]. An FDN is a feedback comb filter with N feedback channels where the delay line and feedback gain elements are replaced with matrices. The delay line lengths are kept mutually prime to maximise the pseudo-randomness of the reverberators.

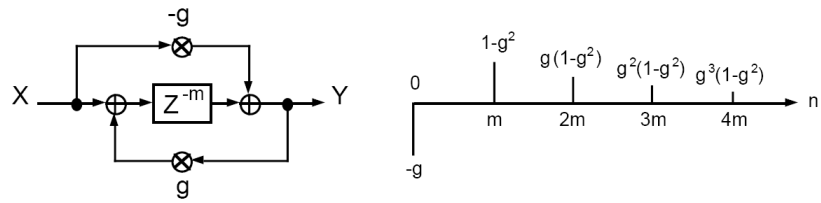


Figure 9.5.2a: Block diagram and Impulse response of an all-pass filter.

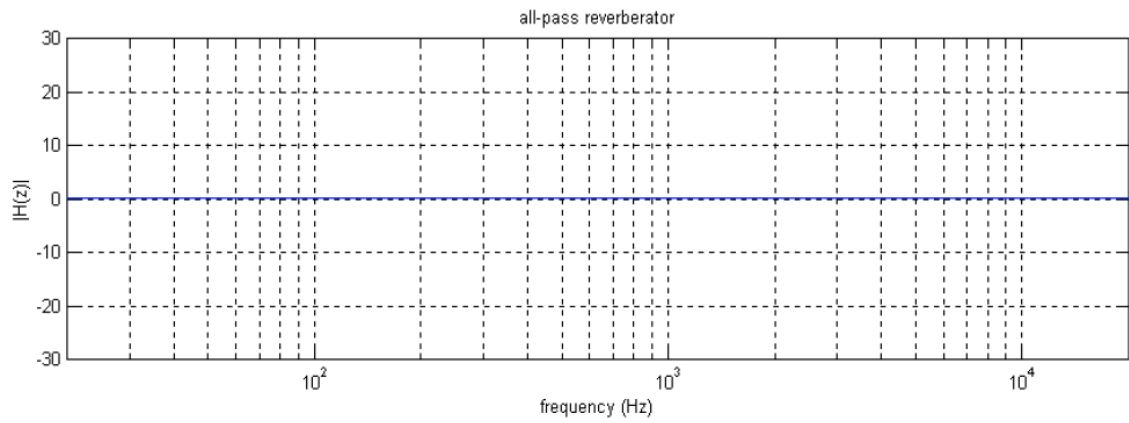


Figure 9.5.2b: Frequency response of an all-pass filter.

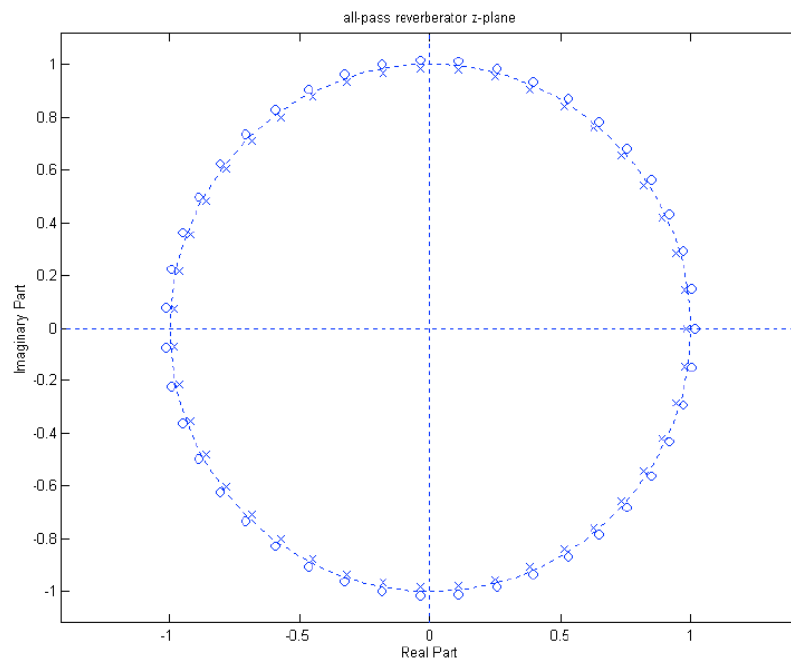


Figure 9.5.2c: Frequency response in z -plane.

The delay line element is represented in the time domain by the matrix,

$$\begin{bmatrix} x_1(n - M_1) \\ x_2(n - M_2) \\ x_3(n - M_3) \end{bmatrix}$$

The feedback line element is represented as the product of a diagonal matrix of the gain values G , and an orthogonal matrix known as the *mixing matrix*, Q :

$$\begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix} \begin{bmatrix} g_1 & 0 & 0 \\ 0 & g_2 & 0 \\ 0 & 0 & g_3 \end{bmatrix}$$

where

$$G = \begin{bmatrix} g_1 & 0 & 0 \\ 0 & g_2 & 0 \\ 0 & 0 & g_3 \end{bmatrix}$$

and

$$Q = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix}$$

The mixing matrix, Q , provides diffusion by distributing the energy of the system between the N channels.

The inner loop calculations of the FDN are described as

$$\begin{bmatrix} x_1(n) \\ x_2(n) \\ x_3(n) \end{bmatrix} = \begin{bmatrix} g_1 & 0 & 0 \\ 0 & g_2 & 0 \\ 0 & 0 & g_3 \end{bmatrix} \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix} \begin{bmatrix} x_1(n - M_1) \\ x_2(n - M_2) \\ x_3(n - M_3) \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} u(n)$$

and the loop output by

$$\begin{bmatrix} y_1(n) \\ y_2(n) \\ y_3(n) \end{bmatrix} = \begin{bmatrix} x_1(n - M_1) \\ x_2(n - M_2) \\ x_3(n - M_3) \end{bmatrix}$$

Jot used this concept to create an ADR algorithm which has individual RTs for separate frequency bands, promoting the absence of tonal coloration in the late reflections, as well as being able to be specified in terms of desired reverberation and frequency response.

A tonal correction filter $E(z)$ is added after the FDN output $v(n)$, where

$$v(n) = \begin{bmatrix} c_1 & c_2 & c_3 \end{bmatrix} \begin{bmatrix} x_1(n - M_1) \\ x_2(n - M_2) \\ x_3(n - M_3) \end{bmatrix}$$

which serves to equalise modal energy amongst the three frequency bands.